TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A Project Report

On

Detection and Prevention of Citrus Diseases in Oranges using Deep Learning

Submitted By:

Prahlad Acharya (THA077BEI031)

Rijan Pokhrel (THA077BEI034)

Sakar Dahal (THA077BEI037)

Vishal Sigdel (THA077BEI048)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

March, 2025

TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A Project Report

On

Detection and Prevention of Citrus Diseases in Oranges using Deep Learning

Submitted By:

Prahlad Acharya (THA077BEI031)

Rijan Pokhrel (THA077BEI034)

Sakar Dahal (THA077BEI037)

Vishal Sigdel (THA077BEI048)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

In the partial fulfillment for the award of the Bachelor's Degree in Electronics, Communication and Information Engineering

Under the Supervision of

Dr. Ritu Raj Lamsal

March, 2025

DECLARATION

We hereby declare that the report of the project entitled "**Detection and Prevention of Citrus Diseases in Oranges using Deep Learning**" which is being submitted to the **Department of Electronics and Computer Engineering, IOE, Thapathali Campus**, in the partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering in **Electronics, Communication and Information Engineering**, is a bonafide report of the work carried out by us. The materials contained in this report have not been submitted to any University or Institution for the award of any degree and we are the only author of this complete work and no sources other than the listed here have been used in this work.

Prahlad Acharya (Class Roll no: 077/BEI/031) _____

Rijan Pokhrel (Class Roll no: 077/BEI/034) _____

Sakar Dahal (Class Roll no 077/BEI/037) _____

Vishal Sigdel (Class Roll no 077/BEI/048) _____

Date: March, 2025

CERTIFICATE OF APPROVAL

The undersigned certify that they have read and recommended to the **Department of Electronics and Computer Engineering, IOE, Thapathali Campus**, a major project work entitled "**Detection and Prevention of Citrus Diseases in Oranges using Deep Learning**" submitted by **Prahlad Acharya, Rijan Pokhrel, Sakar Dahal and Vishal Sigdel** in partial fulfillment for the award of Bachelor's Degree in Electronics, Communication and Information Engineering. The Project was carried out under special supervision and within the time frame prescribed by the syllabus.

We found the students to be hardworking, skilled and ready to undertake any related work to their field of study and hence we recommend the award of partial fulfillment of Bachelor's degree of Electronics, Communication and Information Engineering.

_____

Project Supervisor
Dr. Ritu Raj Lamsal
PhD, University of Malaga, Spain

_____

External Examiner

_____

Project Co-Ordinator
Er. Sudip Rana
Department of Electronics and Computer Engineering, Thapathali Campus

_____

Er. Umesh Kanta Ghimire
Head of the Department,
Department of Electronics and Computer Engineering, Thapathali Campus

March 2025

COPYRIGHT

ACKNOWLEDGEMENT

We extend our sincere gratitude to the Department of Electronics and Computer Engineering for their unwavering support and encouragement throughout the project, "**Detection and Prevention of Citrus Diseases in Oranges using Deep Learning**".

We would also like to express our anticipation and appreciation for the guidance being provided by the supervisor **Dr. Ritu Raj Lamsal**. His expertise and insights are invaluable, and we are thankful for his mentorship throughout the project.

We would also like to thank Dr. Umesh Acharya, Chief Scientist of the Nepal Commercial Crop Research Centre for his insights and suggestion during initial planning of the project. Similarly, we would like to thank District Agriculture Development Office, Kavre for their support during dataset collection. Moreover, we would also like to thank Mrs. Richa Giri, Biotechnologist at Warm Temperate Horticulture Center, NCFD, Kirtipur and her team for their support during testing and evaluation of mobile application.

The department's commitment to fostering innovative projects and providing a conducive learning environment has played a pivotal role in shaping this endeavor. We are grateful for the resources, facilities, and opportunities afforded by the department, which have been instrumental in the project.

Thank you for believing in the potential of this project and for the continuous support provided by the department and supervisor. We are enthusiastic about the development of the Detection and Prevention of Citrus Diseases in Oranges using Deep Learning.

Prahlad Acharya (THA077BEI031)
Rijan Pokhrel (THA077BEI034)
Sakar Dahal (THA077BEI037)
Vishal Sigdel (THA077BEI048)

ABSTRACT

Citrus farming, particularly Mandarin orange cultivation, is a crucial economic activity in Nepal's hilly regions. However, disease detection and management remain major challenges. This study presents an efficient method for identifying and controlling five key citrus diseases affecting the Nepali orange market: Black Spot, Canker, Huanglongbing (HLB), Leaf Miner, and Sooty Mold. We employ the MobileNetV2 model for disease prediction and a One-Class SVM model for initial leaf classification. Additionally, we integrate a Llama-3.2-11b-vision RAG-based chatbot, which analyzes leaf images and provides real-time guidance on disease prevention and orchard management. A mobile application has been developed to integrate the chatbot with a user-friendly interface, making it accessible for farmers. Our approach achieves 95.6% accuracy in disease identification and 85.6% accuracy in orange leaf classification. With its intuitive mobile platform and AI-driven chatbot, this system has the potential to transform citrus farming in Nepal by enabling timely interventions and improved disease management.

Keywords: Citrus, Deep Learning, Huanglongbing, Machine Learning, NLP

Table of Content

List of Figures

List of Tables

List of Abbreviations

2D: Two-Dimensional
BERT: Bidirectional Encoder Representations from Transformers
CNN: Convolutional Neural Network
ConvNets: Convolutional Network
C-SVC: C-Support Vector Classification
FAISS: Facebook AI Similarity Search
FLOPS: Floating Point Operation Per Second
GA-SVM: Genetic Algorithm-Support Vector Machine
GPT: Generative Pre-trained Transformer
GloVe: Global Vectors for Word Representation
HLB: Huanglongbing
HTTP: Hyper Text Transfer Protocol
ILSVR: Incremental Learning Support Vector Regression
Kwargs: keyword arguements
LLM: Large Language Model
LoRA: Low Rank Adaptation
NER: Named Entity Recognition
NLP: Natural Language Processing
PCA: Principal Component Analysis
PCR: Polymerase Chain Reaction
RAG: Retrieval Augmented Generation
ReLU: Rectified Linear Unit
RGB: Red, Green, Blue
RLHF: Reinforced Learning from Human Feedback
SFT: Supervised Fine-tuning
TF lite: TensorFlow lite
t-SNE: t-distributed Stochastic Neighbor Embedding
YOLO: You Only Look Once

1. INTRODUCTION

Citrus fruits boast a long history reaching back thousands of years to Southeast Asia, especially in the areas of present-day India, Myanmar, and China. These regions, housing wild citrus varieties, were the origin of citrus development, leading to essential species such as citron (*Citrus medica*), pomelo (*Citrus maxima*), and mandarin (*Citrus reticulata*). Throughout time, the natural hybridization of these species established the genetic foundation for the majority of contemporary citrus varieties. Nonetheless, ailments such as Citrus Canker, Black Spot, Leaf Miner, Sooty Mold and Huanglongbing present difficulties for citrus agriculture, prompting investigations into disease-resistant cultivars and sustainable methods. In spite of these obstacles, citrus fruits continue to be essential to worldwide agriculture, commerce, and culture. This project mainly focuses on detection of such Citrus Diseases and its precaution in case of an outbreak. An inbuilt chatbot helps in analyzing the samples, detecting the disease, alerting the user, suggesting the possible preventive measures and all the information related to the orange fruit. The mobile app built is feasible for the local farmers and large-scale farmers too for the detection of the disease through the camera in the mobile.

1.1 Background

Effects of Citrus Diseases in Nepal

Citrus cultivation serves as a crucial income stream for Nepalese farmers. Nepali Citrus is a Rs. 2,470 million per year industry. Among all the citrus fruits, mandarin types of citruses occupy 64% and 68% of total citrus growing area and production, respectively. There is a noticeable sharp decline seen in productivity from 12 metric tons a hectare to 8 metric tons in six years [1] due to diseases such as HLB, which inhibits growth and lowers fruit production, along with Black Spot and Citrus Canker, which harm fruit and foliage, have led to significant losses. Fruits of inferior quality command lower prices, impacting both local and export markets. In extreme situations, whole orchards have been destroyed, resulting in minimal to no income for farmers.

The costs have increased due to disease management practices like pesticides, fungicides, and replanting. Farmers frequently encounter financial pressure since young plants require years to grow, causing a postponement in income recovery. Numerous small-scale farmers do not possess the resources needed to effectively tackle these challenges.

The decrease in citrus cultivation has resulted in diminished incomes, job losses, and decreased citrus availability in local markets. Overuse of chemicals for disease control damages soil and ecosystems, possibly leading to pests that are resistant to pesticides. Tackling these problems necessitates government action, investigation into disease-resistant strains, and educating farmers. Integrated pest management (IPM) and sustainable agricultural methods can reduce chemical applications while preserving crop vitality. Joint initiatives among farmers, researchers, and policymakers are crucial for securing the lasting sustainability of Nepal's citrus industry. Some Citrus diseases affecting orange orchards in Nepal are:

Citrus Greening Disease:

Huanglongbing (HLB), often known as citrus greening, is a bacterial infection of citrus plants. *Candidatus Liberibacter* is the bacterium that causes this illness. Sucking insects and plant migration between different countries and continents are two ways that the bacteria propagate over the diverse range of citrus trees. The Asian Citrus Psyllid (*Diaphorina citri*) is an insect that spreads this disease. In citrus trees, the disease hinders the movement of nutrients. Citrus trees are slowly killed by it. The disease initially appears as mottling of the leaves, which is followed by short shoots, a slow death of the branches, and finally small, malformed fruits with bitter juice. It is challenging to detect HLB through visual symptoms as the symptoms observed on leaves and fruits vary and resemble other disorders like micronutrient deficiencies, mainly zinc, iron, and manganese [2].

Figure 1.1: Citrus Greening vs Healthy Leaves

Citrus Canker:

The "Citrus Canker" bacterial disease affects citrus trees and causes harm to crops as well as trade disruptions. It is brought on by the bacteria *Xanthomonas Axonopodis pv. citri*. The signs of citrus canker include yellow halo and raised, brown lesions with water-soaked edges on fruit, leaves, or stems. Citrus Canker primarily causes spots on the leaves and imperfections on the fruit's skin. Insects like aphids and citrus leaf miners that feed on diseased plants can also carry the bacteria, as can water droplets from irrigation or rainfall. The bacterium enters its host through stomata and wounds, from which it invades the intercellular spaces in the apoplast. It produces erumpent corky necrotic lesions often surrounded by a chlorotic halo on the leaves, young stems, and fruits, which causes dark spots, defoliation, reduced photosynthetic rate, rupture of leaf epidermis, dieback, and premature fruit drop in severe cases [3].

Figure 1.2: Citrus Canker infected leaves and fruits

Citrus Sooty Mold

A fungus causes citrus sooty mold, a black, powdery coating that appears on the leaves, stems, and fruit of citrus plants. Mold grows in the sugary waste products known as honeydew left behind by aphids, whiteflies, and scale insects. The species of sooty mold-causing fungi present are determined by a combination of the environment, host, and insect species present. Some sooty mold species are specific to particular plants or insects, while other mold species might colonize many types of surfaces and use honeydew produced by several kinds of insects [4]. The plant is not harmed by the mold itself. This opaque layer may block sunlight, which reduces the plant's ability to use photosynthetic energy and weakens the plant, ultimately resulting in poor development and fruit. One way to help manage the disease is to reduce the number of insects and remove any mold.

Figure 1.3: Sooty Mold infected leaves

Citrus Leaf Miner:

The citrus leaf miner (*Phyllocnistis citrella*) is a tiny moth that’s a real headache for orange farmers. It’s originally from Southeast Asia but has spread to citrus-growing regions worldwide. The trouble comes from its larvae, which tunnel through young leaves, leaving behind squiggly, snake-like trails. This not only makes the leaves look bad but also weakens the plant by messing up its ability to make food through photosynthesis. The larvae mine inside the lower or upper surface of newly expanding leaves, causing them to curl and look distorted. The larvae will not cross their own mine or the mine of another larva. The larvae continue to mine as the leaf fully expands. After the Leaf Miner emerges,

the damage to the leaf caused by mining activity remains. Older leaves that have hardened off are not susceptible unless extremely high populations are present [5].

Figure 1.4: Citrus Leaf Miner infected leaf

Citrus Black Spot

The fungus that causes Citrus Black Spot, also known as *Guignardia citricarpa*, *Phyllosticta citricarpa*, is responsible for the dark, sunken patches on fruit that reduce its marketability and aesthetic appeal. Disease symptoms consist of numerous tiny to middle-sized dark brown to black spots, ranging in size from 3 to 10 mm in diameter. The tiny spots have a grey center with a dark brown to black border surrounded by a green or yellow halo, depending on the maturity of the fruit. In cases of severe infection, the fruits are completely covered with black spots, and rot. The leaves also have numerous black spots [6]. Management strategies include applying fungicides, removing fallen fruit and leaves, and enforcing strict quarantine laws in order to prevent the illness from spreading.

Figure 1.5: Blackspot infected fruit

1.2 Motivation

A number of diseases pose serious threats to citrus crops' health and the industry's capacity to make money. Canker, Scab, Black Spot, Sooty Mold, and HLB (Huanglongbing) are the ones that need to be taken very seriously. Lesions on citrus fruits and leaves caused by canker can weaken the trees and lower the quality of the fruit. A black fungal growth called sooty mold covers leaves, affecting photosynthesis and the general health of the plant. Fruit and leaves that develop rough, elevated sores due to scabs lose some of their appealing aspect and marketability. Dark lesions caused by Black Spot reduce the fruit's market value.

Many people in Nepal or other major citrus-producing nations are unaware about these diseases or know how to prevent them. Traditional detection techniques, such as PCR testing and visual inspection, are labor- and time-intensive and hence not always feasible. In order to tackle these issues, this project suggests creating a mobile application that allows for the identification and treatment of five major citrus diseases: HLB, Canker, Leaf Miner, Sooty Mold, and Black Spot. For the purpose of identifying these illnesses, the application will offer both large-scale growers and nearby farmers an easily navigable interface. A chatbot component will also provide a virtual assistant with useful information on managing illnesses and fruits. This initiative seeks to preserve citrus tree health and maintain the financial stability of the citrus sector by providing farmers with timely, useful tools for disease identification and prevention.

1.3 PCR in Citrus Disease Detection

The process of PCR is pretty fascinating, though it’s not exactly simple. It starts with collecting samples from the plant—usually a leaf or a piece of fruit—then extracting its DNA. Once the DNA is isolated, scientists mix it with special primers, which are tiny bits of genetic code designed to latch onto the DNA of specific pathogens. The whole mixture then goes into a thermal cycler, a machine that rapidly heats and cools the sample to break apart the DNA, copy it, and amplify it into millions of identical pieces. This repeated heating and cooling is what makes PCR so effective—it multiplies even the smallest traces of pathogen DNA until they’re easy to detect. Once the reaction is done, scientists use gel

electrophoresis or fluorescence-based techniques to see if the pathogen is present. It's a highly technical process, but when done right, it provides near-perfect accuracy in disease detection.

The downside is that PCR isn't exactly a quick and easy test. It requires expensive lab equipment, trained specialists, and a controlled environment to avoid contamination. Preparing the samples is time-consuming, and the thermal cycling process alone can take hours. After that, there's still the analysis step, which requires careful interpretation of results. Plus, the chemicals and reagents used in PCR aren't cheap, making it a costly procedure to run regularly. This is why it's not something farmers can just do in their fields—it's a lab-based test that requires skill and patience. Even so, despite all the effort and cost, citrus growers and researchers keep coming back to PCR because nothing else comes close to its accuracy.

At the end of the day, PCR is the gold standard for detecting citrus diseases because it's the only method that truly works at an early stage. Many citrus pathogens can lurk inside a tree for months or even years before showing any symptoms, making visual diagnosis unreliable. Other testing methods, like antibody-based detection, aren't always precise because some pathogens have strains that are hard to tell apart. PCR solves this problem by going straight to the source—the DNA—ensuring that even tiny amounts of a pathogen are caught before they have a chance to spread. This early detection can mean the difference between saving an orchard or losing an entire crop. For citrus farmers, researchers, and regulatory agencies, PCR isn't just a fancy lab technique—it's an essential tool in the fight to protect citrus trees from devastating diseases [7].

1.4 Citrus Diseases in context of Nepal

To understand the condition of Citrus diseases in context of Nepal, the project was discussed with **Dr. Umesh Acharya**, Chief Scientist of the Nepal Commercial Crop Research Centre, to make it applicable in the context of Nepal. During the meeting, the following points were discussed:

- Scope and effectiveness of the project among the Nepali farmers

- Feasibility study of the project
- Extension of the project domain from the detection of HLB disease to include additional orange diseases: Citrus Canker, Citrus Sooty Mold, Citrus Black Spot, and Citrus Leaf Miner/ Citrus Scab (Whichever possible)
- Possibility of local dataset collection in the Sindhuli, Dhulikhel and Kathmandu district

Moreover, we visited orange orchard in Bhattidanda, Dhulikhel to further collect local datasets of orange leaves and explore the natural habitat of oranges. We photographed orange leaves at the farm, focusing on healthy leaves, with some showing signs of sooty mold infection. Additionally, we gained on-field experience at the orange farm, which will contribute to making our project more applicable. During the visit, we also visited District Agriculture Development Office, Kavre to research orange diseases and the government's role in helping farmers tackle them. The following are the findings and are also added to our chatbot's knowledge base:

- In case of infections, farmers may receive full/partial technical support or just technical advice, depending on the severity of the infection.
- Upon detecting an infection, farmers can immediately seek assistance from the District Agriculture Development Office.
- Some trees had healthy leaves but yielded very few fruits, attributed to insufficient irrigation and fertilizers.
- Yellowing of some leaves was observed due to nutrient deficiencies caused by a lack of fertilizers.
- Even after harvesting fruits, farmers are advised to irrigate the land and provide fertilizers to enhance soil fertility.
- The farmers should procure seeds only from government-licensed nurseries.

1.5 Problem definition

The citrus diseases pose a severe threat to the orange production globally. The bacterial disease spread affects sectors of production and economy eventually affects the livelihood of farmers and citrus industry. Effective detection of such diseases is crucial, yet the current

methods are not adequate because the symptoms go unnoticed until the disease has progressed significantly. Traditional techniques like PCR testing requires expert knowledge and laboratory testing, which are both time consuming and expensive, making them unsuitable for the farmers all over the world. Also spraying insecticides only and removing infected trees haven't been very effective in stopping the spread of such diseases. So, there is an urgent need for an effective and automated disease detecting system to save the orange orchards.

This project aims to address all of these challenges by developing a system that uses the advanced technologies like deep learning for the image analysis that can quickly and effectively identify infected plants from images of the leaves facilitating better management of the disease in orchards and timely intervention before the disease spreads significantly. Also, a proper chatbot aims to provide a virtual assistant to the farmers about the diseases and the preventions that can be taken to prevent the spreading of these diseases.

1.6 Objective

The main objectives of the project are:

- To train a model for detection of citrus diseases in Orange using deep learning.
- To create a mobile application that assists in detection of Citrus diseases and suggests proper measures for timely prevention.
- To develop a chatbot as a virtual assistant that can help in analyzing, detecting, alerting, and preventing Citrus diseases in Orange.

1.7 Scope and application

The project aims to address the issue of devastating citrus diseases that poses dangerous threats to orange orchards worldwide including Nepal. By using the advanced image processing technology, the project facilitates at detection and prevention of citrus diseases by enhancing citrus production. The creation of an integrated mobile application with a built-in chatbot is part of the project's scope. The mobile app is equipped with the deep learning model.

Additionally, the system's data collection and analytics enable collaborative research and knowledge sharing among the agricultural community. By aggregating data from multiple sources, researchers could gain insights into disease patterns, environmental factors, and best management practices, driving sustainable prevention strategies. The scalable architecture allows future expansions and integrations with third-party services or systems. For example, this system could interface with weather data providers, soil sensor networks, or drone monitoring systems, providing a comprehensive precision agriculture solution.

In summary, while the immediate goal targets citrus diseases, the project's scope encompasses precision agriculture and crop disease management. Its interdisciplinary nature, data-driven approach, and scalability positions it as a versatile platform with applications beyond the initial use case, fostering innovation, collaboration, and sustainable practices in agriculture.

1.7.1 Project capabilities

- Prevention and detection of the citrus disease in orange plants
- Virtual assistant to answer the queries and assist the farmers

1.7.2 Project Limitations

- The answering capability of the chatbot is limited.
- Limited only for common citrus diseases in Nepal.
- Limited only to orange plants

1.7.3 Project Application

- **Detection:** It can be used for detection of citrus diseases in orange plants.
- **Monitoring:** This project can be used in regular monitoring of the orchards.
- **Prevention:** This project can also be used in preventing the citrus diseases in orange by providing necessary resources and aids.

2. LITERATURE REVIEW

In paper [8], the author developed a framework to help prevent the economy boosting fruit of Pakistan, Guava which is similar to scope of this project. The diseases were usually detected and identified by visual observation; thus, automatic detection is required to assist formers. In this research, a new technique was created to detect guava plant diseases using image processing techniques and computer vision. An automated system was developed to support farmers to identify major diseases in guava. They collected healthy and unhealthy images of different guava diseases from the field. Disease classification was carried out using multiple classifiers, including cubic support vector machine, Fine K-nearest neighbor (F-KNN), Bagged Tree and RUSBoosted Tree algorithms and achieved 100% accuracy for the diagnosis of fruit flies' disease using Bagged Tree. The findings indicated that cubic support vector machines (C-SVM) were the best classifier for all guava disease

In paper [9], an integrated approach was used to suggest a convolutional neural networks (CNNs) model. The proposed CNN model is intended to differentiate healthy fruits and leaves from fruits/leaves with common citrus diseases such as black spot, canker, scab, greening, and Melanose. The proposed CNN model extracts complementary discriminative features by integrating multiple layers. The CNN model was checked against many state-of-the-art deep learning models on the Citrus and PlantVillage datasets. According to the experimental results, the CNN Model outperforms the competitors in a variety of measurement metrics. The CNN Model has a test accuracy of 94.55 percent, making it a valuable decision support tool for farmers looking to classify citrus fruit/leaf diseases.

In paper [10], Deep learning methods was for plant leaf disease detection using CNN activation maps. This research also provided the research progress of deep learning technology in the field of crop leaf disease identification in recent years. Current trends and challenges for the detection of plant leaf disease was presented using deep learning and advanced imaging techniques.

In paper [11], deep learning-based method was used for detecting citrus HLB using YOLOv5l model by the use of digital images. Three models Yolov5l-HLB1, Yolov5l-

HLB2, and Yolov5l-HLB3 were used to analyze the healthy and symptomatic citrus leaves. The micro F1 score using Yolov5l-HLB2 model was calculated to be 85.19% recognizing different symptoms of HLB providing the best result among three models. The model developed can be employed as the preliminary screening tool before the collection of the field samples for the subsequent PCR testing.

In paper [12], the authors claimed that the using of the generic algorithms for feature extraction in machine learning helps in detecting HLB in citrus fruits. The dataset containing the images of citrus leaves and fruits is pre-processed by applying spatial and morphological filters to improve the image quality and reduce the noise. With the help of clustering techniques, the region of interest of the image, the lesion areas is extracted. Taking texture and geometric features from segmented lesion images, hybrid vector was developed by combining color, texture and geometric features. Then using Genetic Algorithm, most features vectors were selected decreasing the feature vector size. At last, the model was trained by reduced featured vector. After the testing and evaluation of the model, the accuracy was calculated to be 90.40% using GA-SVM test scenario.

In paper [13], visible spectrum imaging and C-SVC were used for the detection of HLB in citrus fruits. Different classes of visible images of citrus leaves (nutrition deficient, healthy, HLB and etiolated) under the natural light were collected as the samples and preprocessed to extract the textures and histograms of image color space. Then the feature modelling and recognition of the existence of the HLB was carried out based on C-SVC. This experiment shows the method achieved the accuracy about 91.93% for HLB recognition with low-cost and low-complexity. This paper focuses on developing the cost effective HLB detection mechanism by using the visible spectrum image of the citrus leaves. A feature called local binary pattern is used in S-SVC. Also, the concept of PCA for dimension reduction was used so the training of the model becomes faster.

In paper [14], multi-model fusion network combining RGB image network and hyperspectral band extraction network were used to recognize HLB from four different categories (HLB, suspected HLB, Zinc-deficient and healthy). This network has claimed to effectively classify the above four types of citrus leaves and is better that single-modal

classifiers. The highest accuracy of multi-model network recognition was calculated to be 97.89% while the accuracy achieved using single-modal network with RGB image and hyperspectral image were 87.98% and 89% respectively. For the feature extraction from RGB images, ResNeXt101 was used whereas for hyperspectral image, a simple neural network for feature extraction among the 300–1070-nm hyperspectral data was used. Then the 1D hyperspectral band information and 3D RGB picture information were fused with three different methods namely feature addition, feature multiplication and feature bilinear fusion resulting the accuracy of 94.58%, 93.85% and 95.1% respectively. Then the F1 score for 4 different categories were calculated which concluded that the multi-modal network can effectively help to improve the prediction of HLB in citrus fruits.

In paper [15], the developed processing scheme consists of four main phases. Identification of mostly green colored pixels was followed br masking these pixels based on specific threshold values calculated using Otsu's method. Then the pixels with zeros red, green and blue values and the pixels on the boundaries of the infected cluster (object) were completely removed. The experimental results demonstrated that the proposed technique was a robust technique for the detection of plant leaves diseases. The developed algorithm's efficiency could successfully detect and classify the examined diseases with a precision between 83% and 94%, and could achieve 20% speedup over the approach of threshold selection method from gray-level histograms.

In paper [16], an adaptive approach for the identification of the fruit diseases was proposed and experimentally validated. The image processing based proposed approach was composed of the following main steps; in the first step K-Means clustering technique was used for the defect segmentation, in the second step some state-of-the-art features were extracted from the segmented image, and finally images were classified into one of the classes by using a Multi-class Support Vector Machine. The diseases of apple were considered as a test case and apple scab, apple blotch and apple rot were evaluated. The classification accuracy for the proposed solution was achieved up to 93%.

In paper [17], authors proposed a novel plant leaf disease identification model based on a deep convolutional neural network (Deep CNN). The Deep CNN model was trained using

an open dataset with 39 different classes of plant leaves and background images. Six types of data augmentation methods were used: image flipping, gamma correction, noise injection, principal component analysis (PCA) color augmentation, rotation, and scaling. Using data augmentation could increase the performance of the model was observed. The proposed model was trained using different training epochs, batch sizes and dropouts. Compared with popular transfer learning approaches, the proposed model achieved better performance when using the validation data. After an extensive simulation, the proposed model achieved 96.46% classification accuracy.

In paper [18], Deep learning-based model named plant disease detector was used in plant disease detection. The model was able to detect several diseases from plants using pictures of their leaves. Plant disease detection was developed using a neural network. First of all, augmentation was applied on dataset to increase the sample size. Later Convolution Neural Network (CNN) was used with multiple convolution and pooling layers. Plant Village dataset was used to train the model. After training the model, it was tested properly to validate the results. 15% of data from Plant Village data was used for testing purposes and 98.3% accuracy was obtained.

In paper [19]. the authors introduced a mobile application leveraging machine learning to automate the diagnosis of plant leaf diseases. The system employs Convolutional Neural Networks (CNNs) to classify 38 different disease categories. The training, validation, and testing of the CNN model were conducted using an imagery dataset comprising 96,206 images of both healthy and infected plant leaves. The user interface, developed as an Android app, allows farmers to take photos of infected leaves, subsequently displaying the disease category along with a confidence percentage. The system's performance was evaluated through metrics like classification accuracy and processing time, achieving a 94% overall accuracy in recognizing the most prevalent 38 disease classes across 14 crop species.

In paper [20], the authors demonstrate the effectiveness of convolutional neural network (CNN) generated features and machine learning (ML) classifiers in detecting CBS-infected fruits and leaves with canker symptoms. A custom shallow CNN with a radial basis

function support vector machine (RBF SVM) achieved an overall accuracy of 92.1% in classifying fruits with CBS and four other conditions (greasy spot, melanose, wind scar, and marketable). Additionally, a custom Visual Geometry Group 16 (VGG16) with the RBF SVM classified leaves with canker and four other conditions (control, greasy spot, melanoses, and scab) with an overall accuracy of 93%. These preliminary findings highlighted the potential of using hyperspectral imaging (HSI) systems for the automated classification of citrus fruit and leaf diseases, utilizing both shallow and deep CNN-generated features alongside ML classifiers.

3. REQUIREMENT ANALYSIS

3.1 Dataset Requirement

For accurate predictions, the dataset requires images in a specific format. Either a live photo can be captured or an image of a leaf can be uploaded to determine if the plant is infected. It is crucial that the image of the orange leaf is taken after it has been plucked from the tree. Images taken directly from the tree often contain background noise, which significantly reduces the model's prediction accuracy. High levels of background noise in the image compromise the model's ability to make accurate predictions.

3.2 Knowledge Base Requirement

For accurate response from the chatbot, appropriate knowledge base should be created with all possible queries and their response. The data in knowledge base has to be accurate as the model follows the idea of Garbage In Garbage Out. For accurate response, the data has to be proper and clean. The data is collected from the government booklets, related websites and also by interviewing the personnels from this field. The data is then entered and formatted such that it meets the input requirement of the model.

3.3 Project Requirement

3.3.1 Kaggle

Kaggle is a widely-used online platform for data science and machine learning projects. It offers a vast array of datasets and collaboration tools for data scientists and enthusiasts. With its browser-based interface, Kaggle provides built-in tools and libraries that enable users to write, execute, and collaborate on code effortlessly. The platform supports a rich ecosystem where users can explore, analyze, and model data to address real-world challenges. A notable benefit of Kaggle is its integration with cloud-based GPU resources. Through its kernel feature, Kaggle grants access to GPUs, allowing users to harness powerful computational capabilities without needing costly local hardware.

3.3.2 Jupyter Notebook

Jupyter Notebook is a robust tool that integrates code execution, rich text support, data visualization, and collaboration features within an interactive web-based platform. It enables users to create documents known as notebooks, which consist of cells capable of holding executable code, explanatory text, equations, images, and interactive visualizations. This combination of code and text simplifies the documentation and explanation of ongoing analyses. Additionally, Jupyter Notebook fosters collaboration by allowing users to share their notebooks, facilitating seamless teamwork on data analysis projects.

3.3.3 Google Colab

Google Colab is a popular cloud-based platform for data science and machine learning projects. It provides a comprehensive environment where users can write, execute, and share code effortlessly. With its notebook interface, Colab supports a wide range of libraries and tools, enabling users to perform data exploration, analysis, and modeling. One of the key advantages of Google Colab is its integration with cloud-based GPU resources. Users have access to powerful GPUs, which enhances computational capabilities without the need for expensive local hardware. This makes it an excellent choice for tackling complex machine learning tasks and collaborating on projects with ease.

3.3.4 Python

Python serves as the primary programming language for the project due to its extensive libraries and frameworks for machine learning, deep learning, and natural language processing tasks. Its simplicity and readability make it an ideal choice for rapid prototyping and development.

3.3.5 Convolutional Neural Network (CNN)

Convolutional Neural Networks are part of the class called deep learning processes and they are most applicable to image classification. CNN architecture is inspired by the mechanism of visual processing in the human brain, whereby a network uses the concept

of having multiple layers for learning, as well as extracting hierarchical features from input images. There are different types of layers that make up CNNs, but the most common are those with convolution, pooling, and fully connected layers. The layers within the CNN architecture contribute to the mechanism for improving performance or results.

3.3.6 Flutter

Flutter is a versatile framework for mobile app development that simplifies the creation of high-quality native interfaces for both iOS and Android platforms using a single codebase. With its expressive UI and flexible design, Flutter enables developers to craft visually appealing interfaces that adhere to platform-specific design guidelines. Its reactive programming paradigm facilitates the creation of dynamic and interactive user experiences, while the hot reload feature allows for instant visualization of code changes without restarting the application, fostering a rapid development cycle conducive to experimentation and iteration. Additionally, Flutter offers an extensive collection of pre-built widgets and libraries, streamlining common app development tasks and ensuring near-native performance through performance optimizations that compile code directly to native machine code, making it an ideal choice for building mobile apps prioritizing aesthetics, performance, and cross-platform consistency.

3.3.7 Botpress

Botpress is a conventional AI platform that allows to create various kind of chatbot generally for customer services. It provides an easy interface to create a chatbot without heavy coding and programming. Botpress studio provides an easy flowchart-like approach to building a chatbot. The chatbot created can easily be integrated into websites and mobile API. This is mainly useful for beginners and also to learn basic workings of AI chatbot.

3.3.8 Django

Django is an open-source, high-level Python web framework that facilitates the development of dependable, scalable web applications. By separating the data (Model), business logic (View), and user interface (Template), it follows to the Model-View-

Template (MVT) architecture, which encourages clean, maintainable code. Django comes with an admin interface, security measures (such protections against XSS and SQL injection), authentication, and an ORM (Object-Relational Mapping) for database administration. Django provides developers with the tools they need to quickly and efficiently design apps without having to start from scratch. For projects of all sizes, from big programs to little websites.

3.3.9 Azure Services

Azure is a cloud computing platform developed by Microsoft. It offers a wide range of cloud services such as computing, networking, storage, hosting and AI. These services are hosted in global network of Microsoft datacenters. Azure service provides a framework for developing and running apps in the cloud.

3.4 Feasibility Analysis

3.4.1 Economic Feasibility

The system utilizes the cost-effective resources and a mobile phone which is in hand of almost everyone nowadays ensuring economic feasibility with the focus on minimizing budget requirements.

3.4.2 Technical Feasibility

Use of the well documented technologies, Python programming for the monitoring of the plants enhances the technical feasibility. Integration and comparison of different models is a key consideration.

3.4.3 Operational Feasibility

The project reduces the operating cost as the expensive methods can be replaced with a simple mobile application for the detection of the disease in the orchard. User friendly interfaces facilitate the ease of operation. Moreover, Virtual Assistant greatly aids the user and makes the whole operation easier.

4. SYSTEM ARCHITECTURE AND METHODOLOGY

4.1 Theoretical Review

4.1.1 Deep Learning

Deep learning, a field, within machine learning utilizes neural networks that mimic human like intelligence. These networks consist of layers of neurons that continuously enhance predictions by learning from varied datasets. Their primary advantage lies in processing data enabling them to detect abstract patterns across different types of unstructured data like images, audio, text and videos. At the heart of networks are activation functions, components that introduce nonlinear elements into the network's calculations. These functions play a role in modeling relationships within data allowing the network to accurately link diverse inputs to their respective outputs. By integrating these linear changes neural networks can capture subtle patterns and connections in the data boosting their capacity to tackle complex problems effectively. The effectiveness of learning heavily relies on having access to extensive datasets, for training purposes. The richness and variety of data help the model generalize effectively to real world situations. Continuous progress, in designs like Convolutional Neural Networks (CNNs) for visuals Recurrent Neural Networks (RNNs) for sequences and transformer models, for language tasks significantly boost the impact of learning in different fields.

4.1.2 Convolutional Neural Network

CNNs are one of the artificial neural networks basically developed for the purpose of structured grid data processing, particularly for images. They were basically created by the organization of the animal visual cortex and have almost changed the character of computer vision and image processing. Characterized by the ability to learn spatial hierarchies from input features automatically and adaptively, they make them very strong for image classification, object detection, and semantic segmentation.

A typical CNN architecture includes the following components:

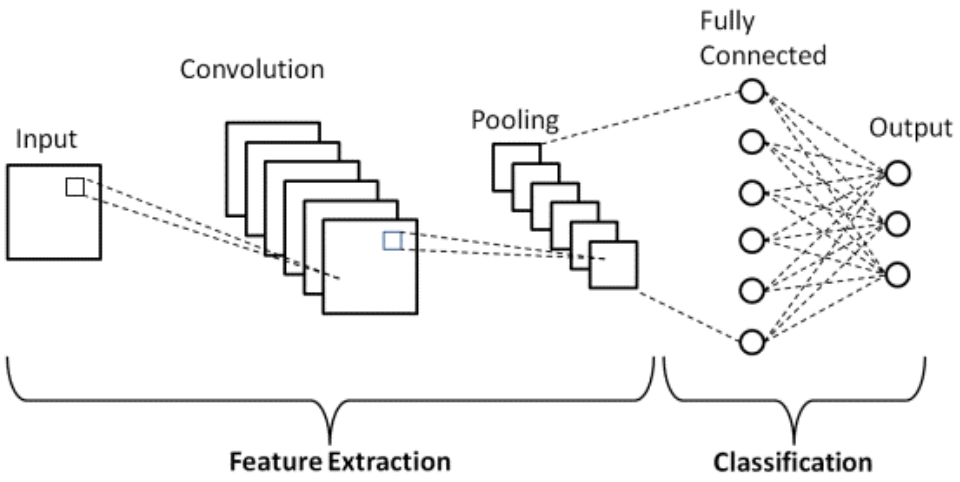


Figure 4.1: Architecture of CNN

a) Convolutional Layers: These form the crux of a CNN. Convolutional layers convolve the input with a set of learnable filters, also called kernels. Every filter is small spatially, along width and height, but extends through the full depth of the input volume. During the forward pass, each filter convolves the width and height of the input volume, computing dot products between entries of the filter and the input at any position. This produces a 2-dimensional activation map that gives the responses of that filter at every spatial position. The network learns filters that activate when they detect some specific type of feature at some spatial position in the input.

b) Activation Functions: After every convolutional layer, an activation function is applied in a nonlinear fashion. The most used activation function in CNNs is the Rectified Linear Unit, ReLU, defined as f(x) = max (0, x). Its advantages are first, computationally very efficient because of sparse activation, and secondly, the likelihood of vanishing gradient problems during training is reduced. Other activation functions, including Leaky ReLU, Parametric ReLU, and Exponential Linear Unit are also used at times.

c) Pooling Layers: Pooling layers are also known as downsampling layers, which help decrease the spatial size of the representation progressively. This reduces the number of parameters and computation in the network and hence controls overfitting. The most

common form of pooling is max-pooling, which carries out partitioning of the input into a set of non-overlapping rectangles and outputs the maximum for each such region.

d) Fully Connected Layers: At the back of multiple convolution and pooling layers, one or more fully connected layers are regularly applied in CNNs. Such layers have a full connectivity to all of the previous layer's activations, just like regular neural networks. The layers gather features that have been extracted by former layers and compose the last output, like class scores.

Moreover, several special-purpose CNN architectures have been designed; some of these incorporate certain types of innovations introduced into them. These include:

a) VGG Networks: It demonstrated that depth is an important factor in neural networks.

b) Inception: It introduced inception modules, which use multiple sizes of filters in parallel.

c) ResNet: It introduced skip connections that allowed training of much deeper networks.

d) DenseNet: Every layer is connected to all other layers in a feed-forward way. This encourages the features to be reused.

e) EfficientNet: This used neural architecture search in optimizing both accuracy and efficiency.

f) MobileNet: MobileNet combines depth wise separable convolutions and multipliers to optimize neural networks for maximum efficiency and performance on mobile and embedded devices.

4.1.3 Performance Metrics

Performance metrics or Evaluation metrics are those parameters that are used to evaluate and analyze the performance of the model while training and testing the model. Some performance metrics are:

Accuracy: Accuracy is defined as ratio of predicted instances to the total number of instances, which indicates overall correctness of the model's predictions. Mathematically,

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$ Equation 4.1

Here, TP = True Positive; TN=True Negative; FP= False positive; FN= False Negative

Precision: Precision is defined as the ratio of true positive predictions to the total number of positive predictions made by the model, which reflects how often the model's positive predictions are correct. Mathematically,

$$Precision = \frac{TP}{TP+FP}$$ Equation 4.2

Here, TP = True Positive; FP= False positive

Recall: Recall is defined as the ratio of true positive predictions to the total number of actual positive instances, measuring how well the model identifies positive cases. Mathematically,

$$Recall = \frac{TP}{TP+FN}$$ Equation 4.3

Here, TP = True Positive; FN= False Negative

F1 score: It is defined as the balanced average of precision and recall, particularly useful for evaluating performance on imbalanced datasets where one class may be more prevalent than the other. Mathematically,

$$F1\ Score = 2 * \frac{Precision*Recall}{Precision+Recall}$$ Equation 4.4

Confusion Matrix: It is a table showing the number of true positives, true negatives, false positives and false negatives.

ROC Curve and AUC: The ROC curve plots the true positive rate against the false positive rate at various threshold settings. The AUC (Area Under the Curve) provides a single measure of overall performance.

4.1.4 MobileNet V2

MobileNet was introduced by the Google Engineers in 2017 which was designed for the efficient deployment of the neural network in mobile devices and the embedded devices. The mobile devices and the embedded devices have the power constraint, memory constraint and mainly the computational constraint. So that the researchers wanted a deep neural network which can work with a smaller number of parameters. With the help of MobileNet, they were able to achieve the remarkable accuracy with a smaller number of parameters [21].

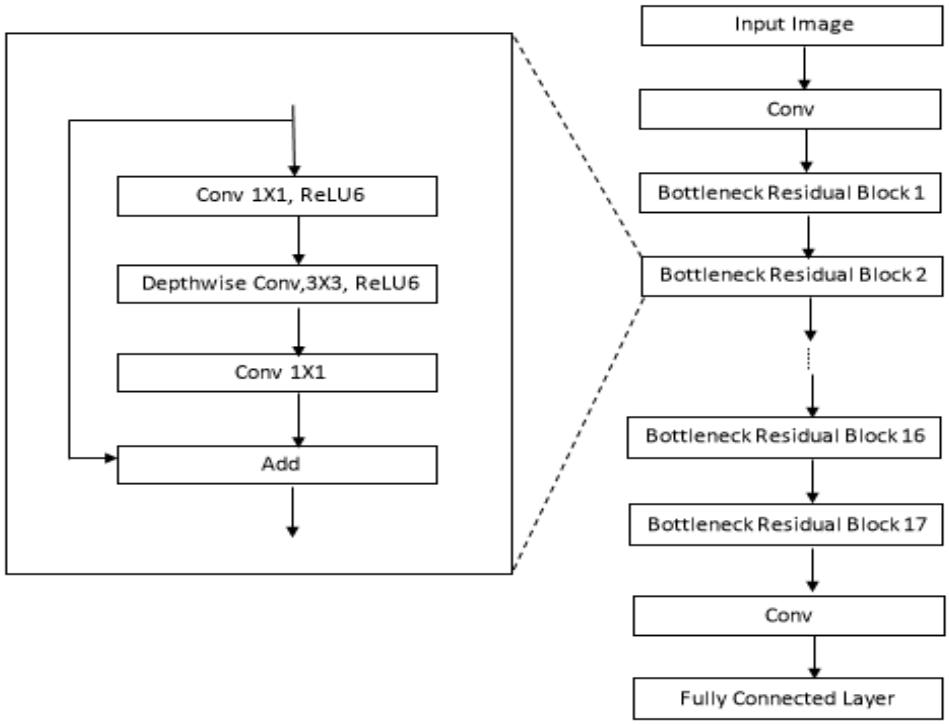


Figure 4.2: Architecture of MobileNetV2

In MobileNetV2 following concepts are used:

- Depth wise Separable Convolutional Neural Network
- Pointwise Convolutional Neural Network
- Width Multiplication
- Resolution Multiplier

Several improvements were done in MobileNetV2 compared to MobileNetV1. The concept of inverted residual block was introduced. These blocks consist of an initial $1\ X\ 1$ pointwise convolution to expand the feature dimensions, followed by a depth wise convolution, and then another $1\ X\ 1$ pointwise convolution to project the features back to a lower dimension. The residual connection is added if the dimension of input and output match. These features were not available in MobileNetV1. By using of inverted residuals and linear bottlenecks, MobileNetV2 significantly reduced the number of parameters and operations compared to its previous version while achieving the higher accuracy. The information is more preserved in MobileNetV2 which improve the representational power and accuracy,

Depth wise Separable Convolutions:

Depth wise Separable convolution is the key building block for MobileNetV2. The basic idea is to replace a full convolutional operator with the factorized version which splits the convolution operations into two different parts. In the first part depthwise convolution is carried out where the convolution operation is carried out by separating the layers of the image and then applying different filters in different layers. In the second part, pointwise convolution is applied where $1\ X\ 1$ convolution is carried out where which is responsible for building the features by computing the linear combinations of the input channels.

In the case of standard convolution, the $h_i\ X\ w_i\ X\ d_i$ input image is taken the convolutional kernel of size $k\ X\ k\ X\ d_i$ is applied across the image. To extract different features from the image, d_j number of filters is applied to get the output as $h_i\ X\ w_i\ X\ d_j$. The standard convolution layers have the computational cost of $h_i\ .\ w_i\ .\ d_i\ .\ d_j\ .\ k\ .\ k$.

Depth wise separable convolution is the replacement for the standard convolution layers. They work almost as well as the regular convolution but only with the cost of $h_i\ .\ w_i\ .\ d_i\ .\ (k^2 + d_j)$ which is the sum of depthwise convolution and $1\ X\ 1$ pointwise convolutions. MobileNetV2 uses $k = 3$ ($3\ X\ 3$ depthwise separable convolution) so the computational cost is 8 to 9 times smaller than of the standard convolution with only small reduction in accuracy.

Performed on ImageNet dataset, the hyperparameters of the model were:

Table 4.1: Hyperparameters of MobileNetV2

Hyperparameters	**Value**
Optimizer	RMSPropOptimizer with Decay = 0.9 and Momentum = 0.9
Activation Function	ReLU6
Weight decay	0.00004
Batch size	96
Learning rate	0.045
Learning rate decay rate	0.98 per epoch
Width multiplier	1
Resolution Multiplier	1
Epochs	200
Dropout Rate	0.2
Expansion Factor	6
Resolution	224 X 224

By increasing the accuracy and efficiency, MobileNetV2 significantly advanced the field of mobile and embedded vision application. Comparing to its previous version, MobileNetV1, the model performed better in ImageNet dataset, using few parameters and requiring the less computational power. The key innovation of the MobileNetV2 lies in the use of Inverted Residual with linear bottleneck making the network efficient.

Use of ReLU6 as activation function in enlarged feature space and a linear bottleneck in the projection layer helps in the information preservation and lowers the possibility of information loss. Performance metrics improved significantly due to this modification in design.

The standard MobileNetV2 had the Top-1 accuracy of 72.0% with 3.4 million parameters. Another version of the network, MobileNetV2 (1.4) which uses the width multiplier of 1.4 obtained the Top-1 accuracy of 74.7% and 6.9 million parameters along with 300 million MAdds.

4.1.5 EfficientNet

EfficientNet is a CNN architecture and scaling method that uniformly scales all dimensions of depth, width and resolution using a compound coefficient. The width refers to the number of layers in the network, the width pertains the number of channels in each layer and the resolution is the input image size.

The original paper of EfficientNet [22] has addressed model scaling and found that carefully balancing network depth, width and resolution could lead to better performance. AT the beginning, all the dimensions of depth, width and resolution were uniformly scaled using simple method to check the performance. The figure:4.3 shows the EfficientNet's Architecture of scaling.

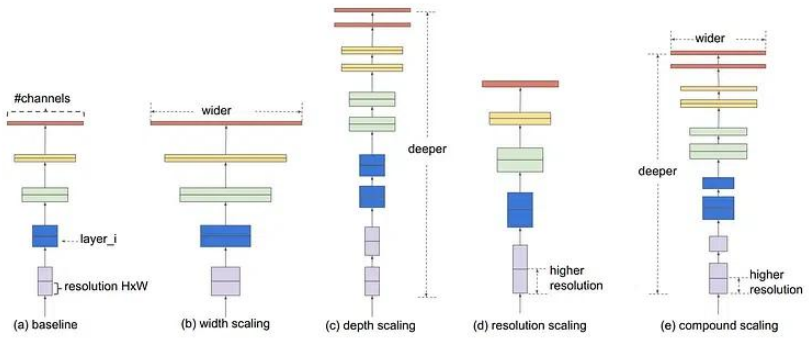


Figure 4.3: Architecture of EfficientNet

The architecture consists of a) baseline b) width scaling c) depth scaling d) resolution scaling e) compound scaling.

In search of a principled method to scale up ConvNets that can achieve better accuracy and efficiency, Compound Scaling was proposed. Compound Scaling makes sense as if input is bigger than the network needs more layers to increase the receptive field and more channels to capture the fine-grained patterns on the bigger image. In the Compound Scaling methods compound coefficient φ was used to uniformly scale the network width, depth and resolution in a principled way:

$$Depth\ (d) = \alpha^{\varphi}$$

$$Width\ (w) = \beta^{\varphi}$$

$$Resolution\ (r) = \gamma^{\varphi}$$

$$s.t.\ \ \alpha.\beta^2.\gamma^2 \approx 2 \qquad \text{Equation 4.5}$$

$$\alpha \geq 1, \beta \geq 1, \gamma \geq 1$$

Where α, β, γ are constants that can be determined by a small grid search. Intuitively, ϕ is a user-specified coefficient that controls how many more resources are available for model scaling, while α, β, γ specify how to assign these extra resources to network width, depth, and resolution respectively.

Notably, the FLOPS of a regular convolution operation is proportional to d, w^2, r^2. This means:

- Doubling network depth will double FLOPS.
- Doubling network width or resolution will increase FLOPS by four times.

Since convolution operations usually dominate the computation cost in ConvNets, scaling a ConvNet with the given equation 4.5 will approximately increase total FLOPS by $(\alpha \cdot \beta^2 \cdot \gamma^2)^{\phi}$

$\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ is constrained such that for any new ϕ, the total FLOPS will approximately increase by 2^{ϕ}. When $\phi=1$, assuming twice more resources available, and do a small grid search of α, β, γ the best values for EfficientNet-B0 were $\alpha = 1.2$, $\beta = 1.1$, $\gamma = 1.15$ under the constraint $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$. This then empirically quantifies the relationship between the three dimensions of the network width, depth and resolution.

There were two observations in this experiment,

Observation-1: Scaling up any dimension of network width, depth, or resolution improves accuracy, but the accuracy gain diminishes for bigger models.

Observation-2: In order to pursue better accuracy and efficiency, it is critical to balance all dimensions of network width, depth, and resolution during ConvNet scaling.

Performed on the ImageNet Dataset, the hyperparameters of the model were:

Table 4.2: Hyperparameters of EfficientNet

Hyperparameters	**Value**
Network Depth and Width	237 layers and depth multiplier are 1.0
Image Resolution	224 X 224
Width Multiplier	1.0
Activation Function	Silu
Dropout Rate	0.2
Regularization	L2 Regularization

Learning Rate	0.01
Batch Size	256
Epochs Trained	300
Optimizer	RMS prop

The EfficientNet models outperformed other state-of-the-art convolutional neural networks on the ImageNet dataset in terms of both accuracy and efficiency. Specifically, EfficientNet achieved better accuracy with fewer parameters and less computational cost (FLOPS) compared to other models like ResNet, DenseNet, and Inception. This performance improvement was due to the balanced scaling of depth, width, and resolution.

The EfficientNet-B0 had Top-1 Accuracy of 77.1% with 5.3 million parameters and 0.39 billion FLOPS. The largest EfficientNet-B7 surpassed all other models with quite a margin while the B0 model lags a little behind. This project will actually contain the use of EfficientNet-B0 for its easy to use and less parameters to handle as well as for its lightness.

4.1.6 Inception-v3

Inception-v3 is a type of neural network designed to classify images. It builds on the earlier Inception models and was developed by Google in 2015 [23]. The key improvements in this model include the use of factorized convolutions, efficient grid size reduction, and auxiliary classifiers to aid in training. The main features of InceptionV3 are:

Inception Modules: These combine different sizes of convolutional filters and pooling layers to capture various features of the image at multiple scales.
Smaller Convolutions: Instead of using large convolutional filters, InceptionV3 uses smaller ones, which helps to reduce computational demands and increase efficiency.
Auxiliary Classifiers: The model includes additional classifiers to aid in training and enhance convergence.

Batch Normalization: This technique normalizes the input to each layer, which improves training stability and speed.
Fewer Parameters: By using factorized convolutions, InceptionV3 has fewer parameters than its predecessors, making it more efficient.

InceptionV3 can be further understood by the help figure: 4.4. There are multiple layers with optional auxiliary classifier which aids in training and convergence enhancement. Instead of big convolutions, it uses number of smaller convolutions to reduce computational costs and enhance efficiency.

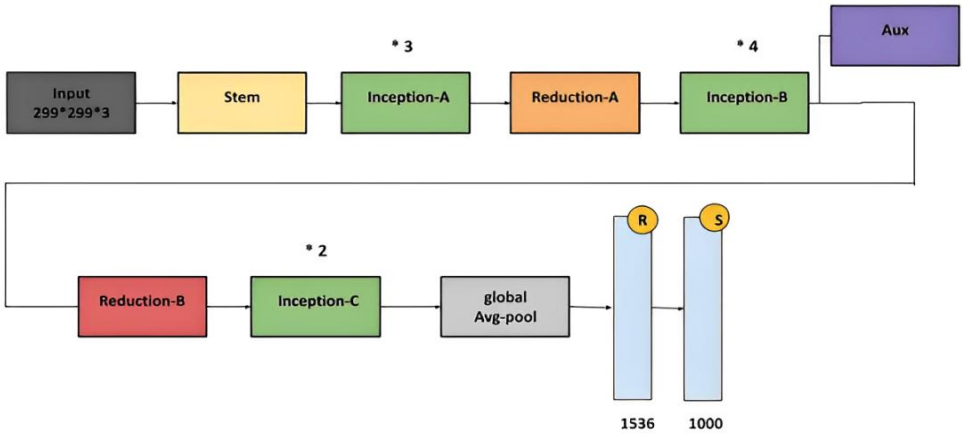


Figure 4.4: Architecture of Inception-v3

The original inception module also had 5x5 convolution, InceptionV3 uses smaller convolutions such that it reduces computational demands and increases efficiency. A 5x5 convolution is replaced by two 3x3 convolutions which sees reduction of parameters. For example: An input with channel C has 5x5xC parameters whereas two 3x3 convolutions have 2x3x3xC parameters.

Moreover, Inception-v3 also uses asymmetric convolutions such as nx1 or 1xn. For example, using a 3 × 1 convolution followed by a 1 × 3 convolution is equivalent to sliding a two-layer network with the same receptive field as in a 3 × 3 convolution. Still the two-

layer solution is 33% cheaper for the same number of output filters, if the number of input and output filters is equal.

The paper proposed several improvements to the Inception architecture, which has been highly successful in computer vision tasks. They have introduced a more efficient version of the model, called Inception-v3, which significantly reduces computational complexity while maintaining high accuracy. The hyperparameters of model were:

Table 4.3: Hyperparameters of Inception-v3

Hyperparameters	**Value**
Optimizer	RMSProp with decay of 0.9 and momentum = 1.0.
Activation Function	ReLU
Weight decay	0.00004
Network depth	48
Batch size	32
Learning rate	0.045
Learning rate decay rate	0.94 per two epoch
Epochs	100
Resolution	299x299

The highest quality version of Inception-v3 reached 21.2%, top-1 and 5.6% top-5 error for single crop evaluation on the ILSVR 2012 classification, setting a new state of the art. This is achieved with relatively modest (2.5×) increase in computational cost as compared to other networks. Moreover, the ensemble of four Inception-v3 models reached 3.5% with multi-crop evaluation reaches 3.5% top5 error which represents an over 25% reduction to

the best published results and is almost half of the error of ILSVRC 2014 winning GoogLeNet ensemble.

4.1.7 One Class SVM

The One-Class Support Vector Machine is a unsupervised learning algorithm which is generally used for anomaly detection. It is generally used in the scenario where training data is primarily from a single class, and the goal is to distinguish between the inliers (data points belonging to the learned class) and outliers (unknown data).

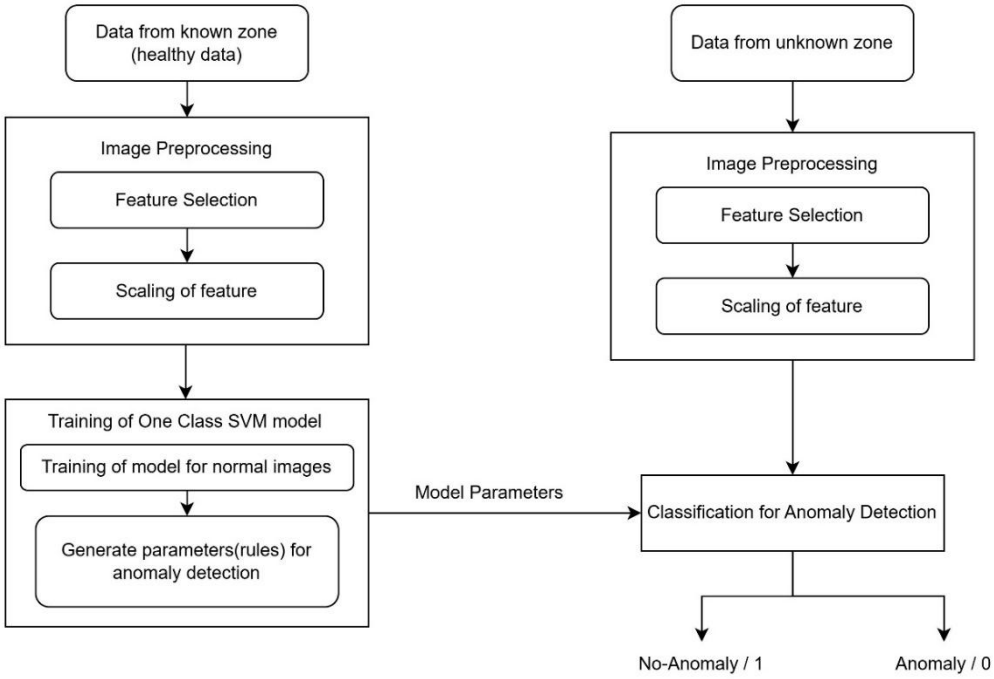


Figure 4.5: Architecture of One Class SVM

One Class SVM creates a boundary in high dimensional space on the basis of the data points of target class. It ensures that the data provided to the model lie within the boundary while any data points outside the boundary are considered anomalies. Mapping of input data into higher dimensional space using kernel function makes the data separable in new space. A decision boundary is created in the transformed space to enclose the majority of training data points, leaving outliers outside the boundary. When the new data is provided to the model, its feature is extracted and projected into same high-dimensional space. The

model then evaluates the position of the model within the decision boundary. If it lies within the boundary, it is classified as belonging to the target class with result=1 and it is flagged as anomaly if it lies outside the boundary with result=0.

One Class SVM is used in the project as the training data predominantly consists of orange leaves, concept of One Class SVM approach is used. It allows the model to effectively classify leaves that do not belong to orange category as outliers.

4.1.8 Transformers

The advent of Transformer models has marked a significant milestone in the field of deep learning, particularly in natural language processing (NLP). Introduced in the seminal 2017 paper "Attention is All You Need" by Vaswani et al., Transformers have revolutionized how machines understand and generate human language. Unlike their predecessors, which relied heavily on recurrent neural networks (RNNs) and convolutional neural networks (CNNs), Transformers utilize a novel architecture based on self-attention mechanisms, allowing them to process input data more efficiently and effectively. At the core of the Transformer architecture is the encoder-decoder structure. The encoder processes input sequences to produce contextualized representations, while the decoder generates output sequences based on these representations. Each encoder layer consists of two main components: a multi-head self-attention mechanism and a feedforward neural network. The self-attention mechanism enables the model to weigh the importance of different words in a sentence, thereby capturing relationships between them regardless of their distance in the sequence. This is achieved through parallel processing, where multiple attention heads operate simultaneously, allowing for a richer understanding of context.

Positional encoding is another critical feature of Transformers. Since these models do not inherently understand the order of tokens in a sequence, positional encodings are added to token embeddings to provide information about their positions. This ensures that the model can maintain an awareness of sequence order while processing data in parallel. The combination of self-attention and positional encoding allows Transformers to excel at tasks such as translation, text summarization, and even image processing when adapted into

Vision Transformers. Moreover, Transformers have found applications across various domains beyond NLP; they are instrumental in real-time translation services, enabling seamless communication across languages. In bioinformatics, they assist researchers in predicting protein structures and understanding genetic sequences, significantly accelerating drug discovery processes. Their ability to analyze large datasets has made them invaluable in fraud detection, recommendation systems, and healthcare analytics.

The sheer scale of Transformer models has also led to the emergence of large language models (LLMs) like GPT-3 and BERT. These models are pre-trained on vast corpora of text data, enabling them to generate coherent and contextually relevant text based on minimal input. The training process leverages self-supervised learning techniques, allowing for improved performance without the need for extensive labeled datasets. As a result, Transformers have become foundational models in AI research, with over 70% of recent AI papers referencing them. Despite their success, Transformers face challenges such as high computational costs and environmental concerns due to energy consumption during training. Innovations like mixture-of-experts (MoE) architectures aim to address these issues by allowing models to utilize only a subset of parameters for specific tasks, thereby reducing resource requirements while maintaining performance.

Transformer models represent a transformative leap in deep learning methodologies. Their unique architecture enables efficient processing of sequential data, leading to advancements across various fields. As research continues to evolve, Transformers are likely to underpin even more sophisticated AI systems that can understand and generate human-like text with unprecedented accuracy and efficiency.

4.1.9 Large Language Models (LLMs)

Large Language Models are complex AI systems that understand and generate human-like text. The inner structure of LLMs is based on deep learning architectures, mainly transformer models, which enable the effective processing of extensive textual material. The basic idea behind LLMs is that they learn the statistical relations within languages while analyzing large corpora. This allows them to predict the next word in a sequence,

making them very good at various tasks related to language: text generation, summarization, translation, and question answering.

The general training of LLMs includes two major phases: pre-training and fine-tuning. In the pre-training phase, LLMs are exposed to some very large dataset without labeled outputs and learn by unsupervised learning techniques to find the patterns and structures in the language. The pre-training phase enables the model to grasp general properties of the language, including grammar, syntax, and semantics. After pre-training, the models are fine-tuned on smaller datasets, which have been task-specific. This enables them to be more specialized in specific applications and improve performance in tasks like sentiment analysis or named entity recognition. Fine-tuning is indispensable because it adapts the ability of the model to respond to specific user needs at the same time as benefiting from the general knowledge learned from pre-training.

LLMs primarily adopt architectures with transformers that have layers pre-equipped with self-attention mechanisms. This allows the model to scale different tokens of a sequence in terms of their relative importance, effectively capturing long-range dependencies and contextual information. Instead of processing sequences sequentially as is the case with traditional RNNs, transformers process entire sequences in parallel, thereby drastically reducing training time and increasing computational efficiency.

While LLMs have indeed shown fantastic capabilities in generating human-like text and solving complex language problems, they are not devoid of their limitations. These models can also pick up several biases from the data they are trained on, which might lead to inaccuracies or inappropriate outputs. Overcoming these challenges is an active area of research as developers strive to enhance the robustness and ethical implications of LLM applications.

4.1.10 Retrieval Augmented Generation (RAG)

RAG or Retrieval-Augmented Generation, is an innovative approach that extends Large Language Models' capabilities by embedding information retrieval mechanisms into the

process of generation. This provides LLMs with an opportunity to refer to knowledge bases for better performance in terms of relevance and accuracy. The architecture of RAG consists of two stages: retrieval and generation. During the retrieval phase, a user query will typically prompt the model to search for relevant documents or data from a pre specified knowledge base. This search usually uses embeddings, or numerical representations of text stored in a vector database, to identify the most relevant information given the context of the query. Once relevant documents are retrieved, they become used in augmenting the original query with additional context that informs the subsequent generation phase.

During the generation, the LLM intertwines the augmented input with internally acquired knowledge from training for a coherent and contextually appropriate response. This duality not only enables LLMs to output more informative results but also mitigates some of the weaknesses of older models, including static knowledge bases that become outdated. By leveraging current and domain-specific information, RAG makes LLMs relevant and reliable in chatbots, customer support systems, and content creation tools, among other applications.

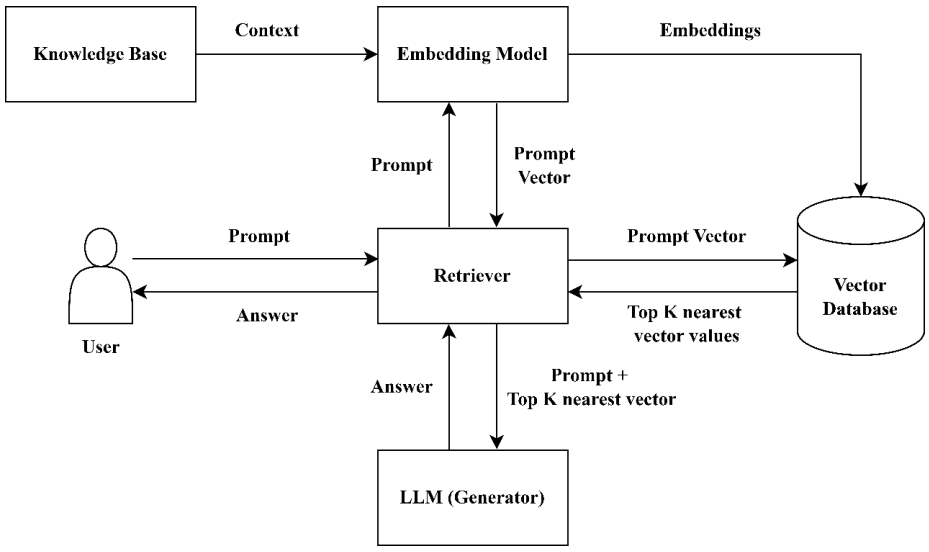


Figure 4.6: System Architecture of RAG

The components of RAG include Knowledge Base, Query, Retriever, LLM, Embedding and Vector database.

A RAG system can be at a glance understood as enhancing a LLM model with required context. The generalized model is given a defined scope which is used as context to generate responses based on the information provided.

i. **Knowledge Base:** The knowledge base is the augmenter of the user query. The assumption of the system is that the user queries specific topic of current scenario and the LLM model might not be trained in the scenario specifically. The knowledge base in this case gives the model required information about the query scenario which is then used to generate a response that satisfies the user and provides the accurate information as well.

ii. **Embedding:** A machine cannot in any way understand how humans perceive languages. There requires a way to make machines adept to human method of languages, for this embedding are used. Embeddings are vector representations of word, phrase or sentences of the model's context. Embeddings are stored as vector which has both a numeric measure and a direction which allows closely occurring words to be placed together which helps preserve and understand context in much detail and in lower dimension. embeddings map words to dense vectors in a lower-dimensional space. This mapping is done in such a way that semantically similar words are closer together in the embedding space. So, embeddings are a way by which texts are converted to numerical values where semantically similar texts are closer together.

iii. **Vector database:** A vector database is a collection of data stored as mathematical representations. Vector databases make it easier for machine learning models to remember previous inputs allowing the models to perform power search as well as text-generation. Vector databases make it possible for computer programs to draw comparisons, identify relationships, and understand context. Each vector in a vector database corresponds to an object or item. These vectors are likely to be lengthy and complex, expressing the location of each object along dozens or even hundreds of dimensions. So, vector database is a spatial storage method that stores embeddings of a deep learning model to be used by it for similarity searches, contextual analysis, generation.

iv. **Retriever:** Unlike traditional search, retrievers combine keyword and vector searches to retrieve relevant documents. They match exact terms through keyword searches and understand the context using vectors, ensuring the most relevant documents are retrieved. The user query is embedded to understand what context and scenario is user referring to, then a semantic search is performed in the vector database to identify and rank the received documents(results). So, retrievers are the brain of the RAG system even though its main component is an LLM. While a LLM is just a generation step, a retriever is a method by which a query is processed, the meaning and query is understood through embedding then the prepared vector store is used to find and rank the possible responses and fed to the LLM/ generator from which user desired response is generated.

v. **Generator:** Generator are the final stage of response generation in a RAG system just before the response is feedbacked based off users' query. A LLM is used as a generator. Using the retriever to pass the question vector to the vector store to obtain top k nearest neighbor vectors. The original question as well the retrieved answers is passed to the generator LLM as prompt. This is used as input using which the LLM understands the context as well the query. The response until this stage are raw which only contain the knowledge base contents, the LLM has its own embeddings which it uses for generative purpose to generate a proper response from the raw texts, adds or improves the scope of the text and then generate a proper finalized answer to return to the retriever which returns the generation back to user.

4.1.11 Finetuning

Finetuning in simple word means making small changes to something in order to make it work as the user desire. Similarly, finetuning a model means to adapt it towards required dataset. A model generally is like jack of all and master of none. Making a model master of a certain field by feeding it required dataset is known as finetuning. At high level, finetuning involves following steps:

1. Prepare and upload training dataset
2. Train a new finetuned model

3. Evaluate results and go back to step 1 if needed
4. Use the finetuned model

The models like Alpaca and Gemma2 can be finetuned to achieve task following characteristics.

Alpaca

Alpaca is an instruction-following language model which is fine-tuned from Meta's LLaMA 7B model. The Alpaca model is trained on 52K instruction-following demonstrations generated in the style of self-instruct using text-davinci-003. On the self-instruct evaluation set, Alpaca shows many behaviors similar to OpenAI's text-davinci-003, but is also small and easy/cheap to reproduce [24]. The Alpaca model is used in instruction following tasks. The dataset required for Alpaca model is organized with examples that include instruction, input and response. Instruction and input give proper context and response is the output expected from the model.

Gemma2

Gemma2 is a updated text-to-text model of Gemma family with newer architecture trained through distillation from larger models. Gemma 2 significantly advances the performance relative to comparable scale open models and are even competitive with some models more than twice their size, across a variety of automated benchmarks and human evaluations. Gemma-2 is based on decoder-only architecture [25]**.** Gemma2 is 2B version of Gemma model with two billion parameters.

Alpaca and Gemma2 model can be used together because alpaca dataset provides instruction following finetuning objective and Gemma2 is a light weight model that can provide a robust backbone for creation of complex text output.

4.2 System Block Diagram

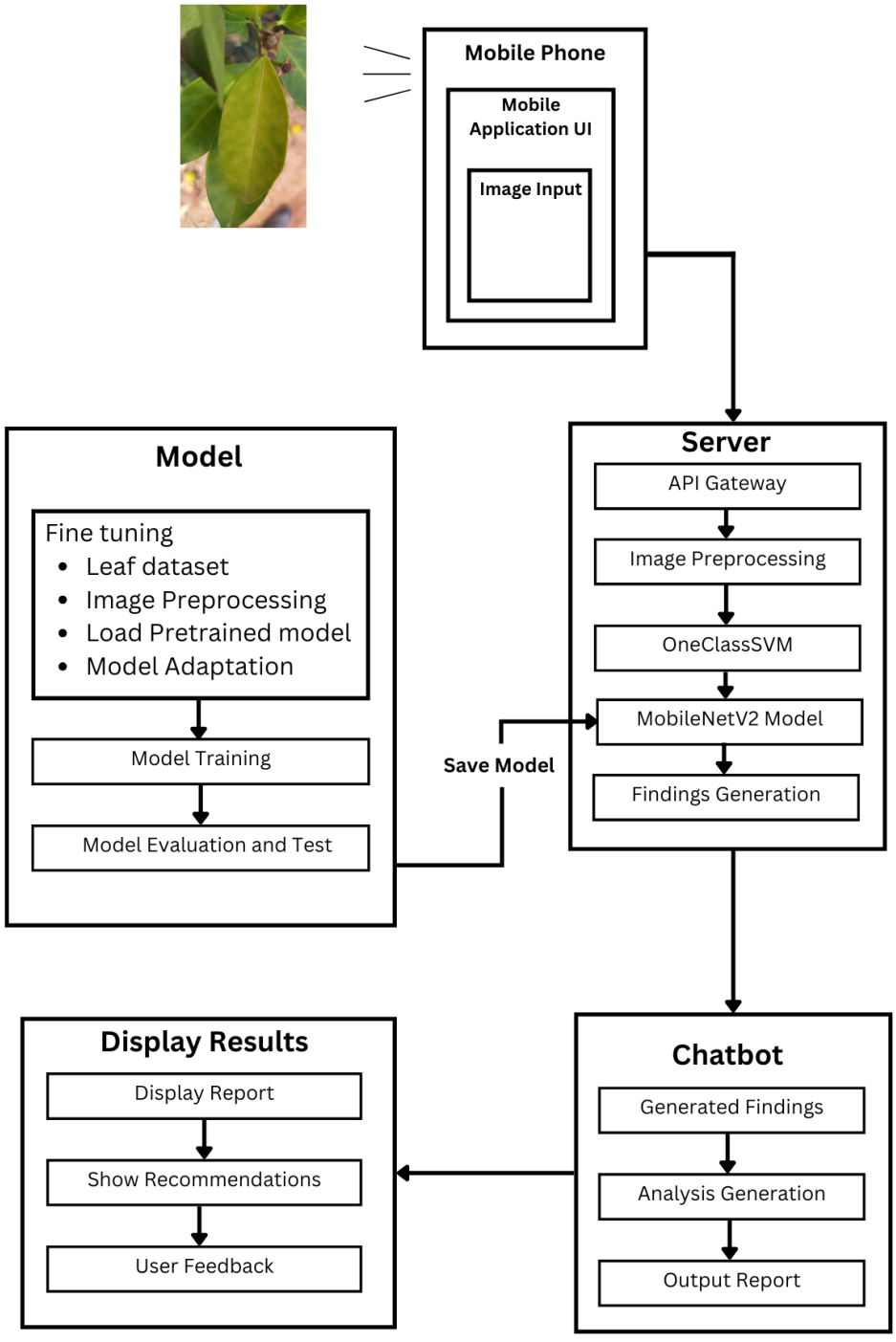


Figure 4.7: System Block Diagram

4.2.1 Mobile Phone

This is the block through which input is fed into the system. A mobile phone camera is utilized for input. A cross-platform mobile application is developed for this purpose. The input is captured through the mobile camera, and the output is displayed on the mobile screen. The mobile application is developed using Flutter. Initially, the UI/UX design was created using Figma, which was then used for the mobile application development. For optimal results, a clear mobile camera should be used.

4.2.2 Server

In the process of efficient and accurate disease prediction from images using a mobile application, initial preprocessing to the final prediction is carried out. In this block, server-side workflow is highlighting the significance of each step and its impact on disease diagnosis.

- **Image Acquisition**

When a user captures and uploads an image via the mobile application, it goes through a chain of events. Here how the image goes through the transformation and analysis is understood.

- **API Gateway**

This is the entry point for the image. This manages the user's request, ensuring secure communication between clients and various services. It acts like a bridge to direct the image to its subsequent stage.

- **Image Preprocessing**

In this part, the raw image is prepared for the model input. At first, the dimension of the image is adjusted to match the MobileNetV2 model's expected input size (224 X 224). Uniform dimension facilitates consistent preprocessing across all the images. After that, the pixel values are adjusted to the normal range from 0 to 1 which helps the neural network to predict effectively.

- **The Neural Network (Model)**

The pre-trained model is a powerful deep learning architecture trained on a variety of datasets to recognize disease-related patterns. During this, the preprocessed image passes through the layers of neurons extracting the features and building a representation of the image. As the model learned from the labeled data during the training process such as textures, properties and shapes related to the disease, the model will generate the prediction indicating if there is presence of citrus disease or not.

- **Disease Detection and Result:**

In this stage, prediction turns into actionable insights. The model classifies the image based on a predefined threshold, indicating the disease if the probability exceeds certain levels. The server then compiles the result and sent back to the frontend, allowing the user to view the diagnosis and take timely actions.

4.2.3 Model

In this block, all the process related to the model takes place. The pre trained model is adapted to required dataset by using an approach known as transfer learning. After transfer learning, training of the model and fine tuning is performed. Then model is evaluated and tested.

- **Transfer learning:**

Transfer learning is a machine learning method where a model trained for one task is repurposed as the foundation for a model on a new task. It takes advantage of the insights gained from solving one problem to address a different but related problem. This technique is especially beneficial when data for the new task is limited, as it allows the model to utilize the extensive data from its initial training.

Transfer Learning mainly involves following steps:

- Choose a pre-trained model
- Load a pre-trained model

- Freeze layers
- Add custom layer
- Compile model
- Evaluate model

- **Model Training**

This is one of the crucial steps for Transfer Learning. After adding and compiling the model, the model should be trained such that it can be adapted to the available dataset. Training is performed for certain epochs. The training is done until the accuracy graph is not plagued generally.

As for fine tuning, it involves small steps to adapt the pre trained model more effectively. It involves steps such as: Unfreezing the model, Adjusting the learning rate, Continuing training and Monitoring. The parameters are needed to be constantly monitored such that the model doesn't underfit or overfit.

- **Model Evaluation and Test**

Model evaluation helps to analyze how well the model performs with unseen data. Model evaluation can be done by the help of Performance metrics such as accuracy, precision, recall and F1 score. Moreover, Model testing involves the model's performance in completely separate test dataset that hasn't been used while training or validating. Model testing is performed with the help of Graphs, confusion matrix, feature maps, etc. Lastly, the model is saved and stored in the server.

4.2.4 Chatbot

In today's conversational era, users prefer interactive discussions to gain deeper insights from model results. Chatbot facilitates this interaction, allowing users to obtain detailed information about diseases and make informed decisions and take necessary actions.

- **User Query Reception**

After the server processes the image and generates findings about the disease, chatbot receives the output. User seek for the clarification, validation and further information related to predicted diseases. For that user queries about the related topics which can vary in complexity and context

- **Preprocessing**

The chatbot breaks down the queries into smaller units called tokens. Each token represents word or sub phrase. The techniques like lemmatization and stemming is carried out which reduce the tokens to their base form. After that the content identifies the user intent-whether it is greeting, inquiry or directive. This sets the tone for upcoming interactions. At last the specific entities are extracted from the query which enhances context-aware responses.

- **Intent Categorization**

The chatbot is equipped with a trained model to categorize the intents. For example:

- Salutation: Simple greetings like “Hi” or “Hello”
- Inquiry: Questions about disease symptoms, treatment or prevention
- Directive: Request for the specific actions like “Show me the details about HLB”

- **Dialog Management**

The chatbot remembers the previous exchanges. If a user tries to refer back to earlier message, the chatbot maintains the context. Similarly, when user’s intents or entities are unclear, the chatbot handles the fallback scenarios with default messages or seek for clarifications.

- **Human-Like Responses**

Before delivering responses to the user, post processing makes sure that it is logical and formatted correctly. This technique guarantees that the chatbot will provide interactions that are relevant, contextual and can mimic the human language.

4.2.5 Display Result

This is the block where final output is displayed to the user. The prediction made by model along with the suggestion from Chatbot is displayed to the user through mobile application. Moreover, the user can give feedbacks and recommendations for improvement of the mobile application.

4.3 Flowchart

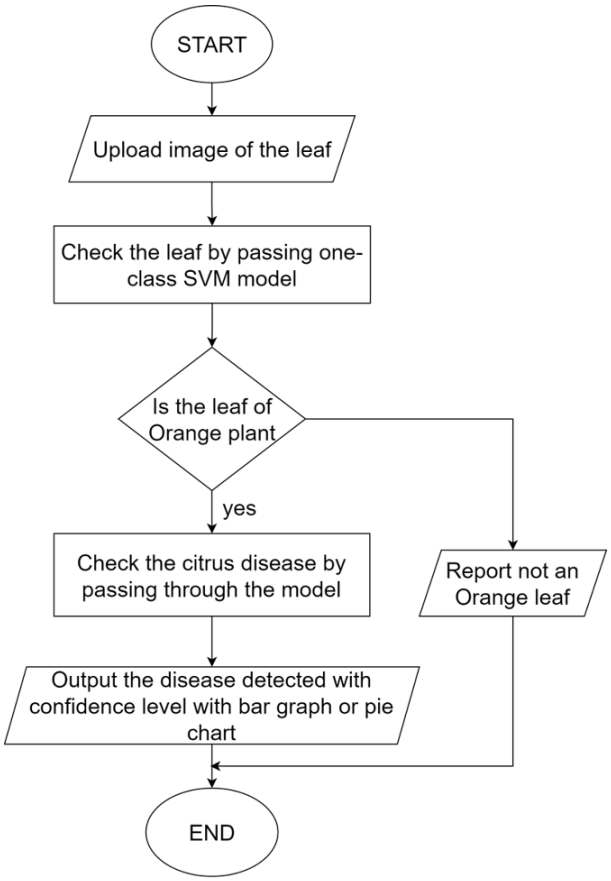


Figure 4.8: Prediction Model flowchart

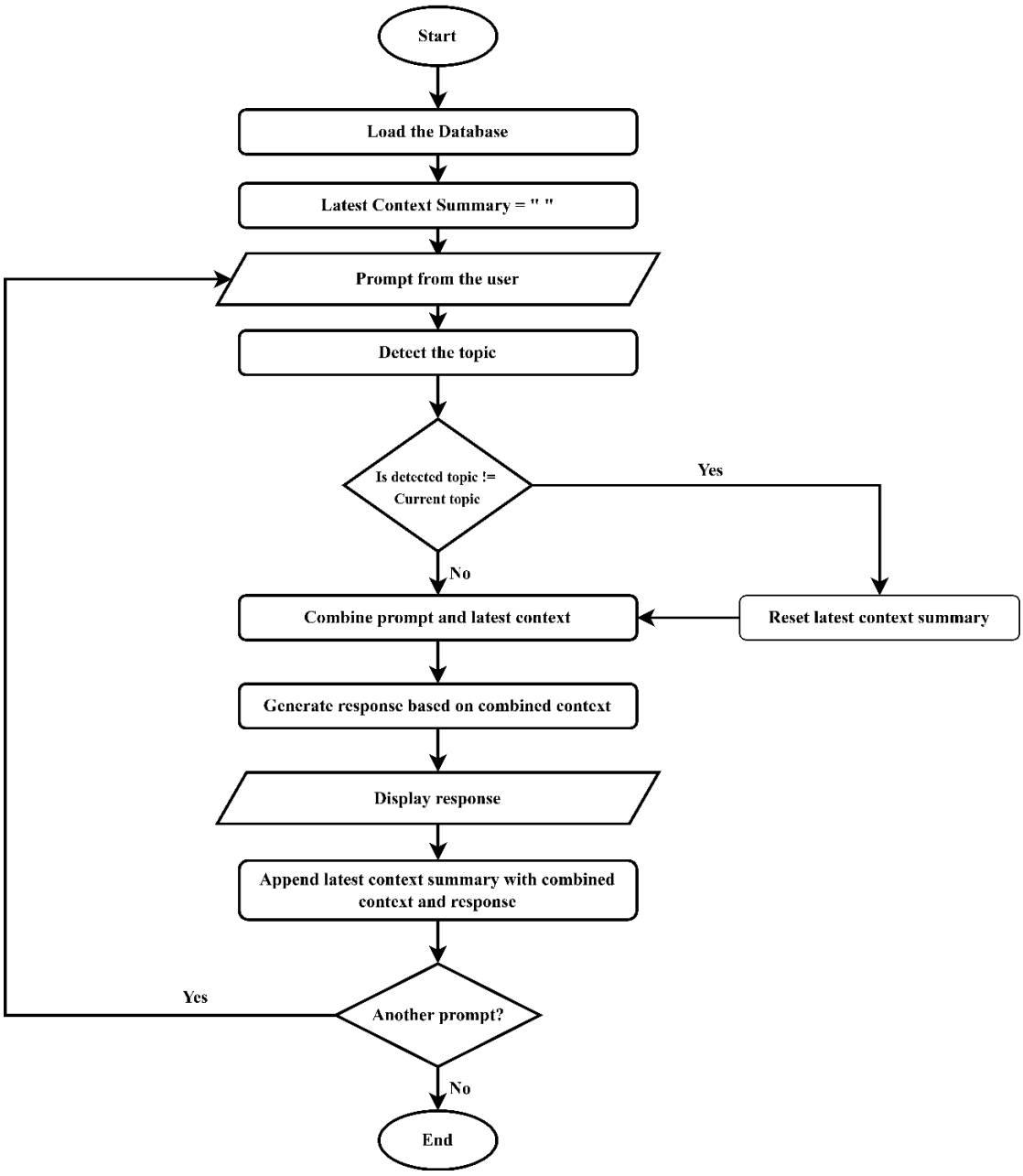


Figure 4.9:Flowchart showing workflow of chatbot

4.4 Server Architecture and Workflow

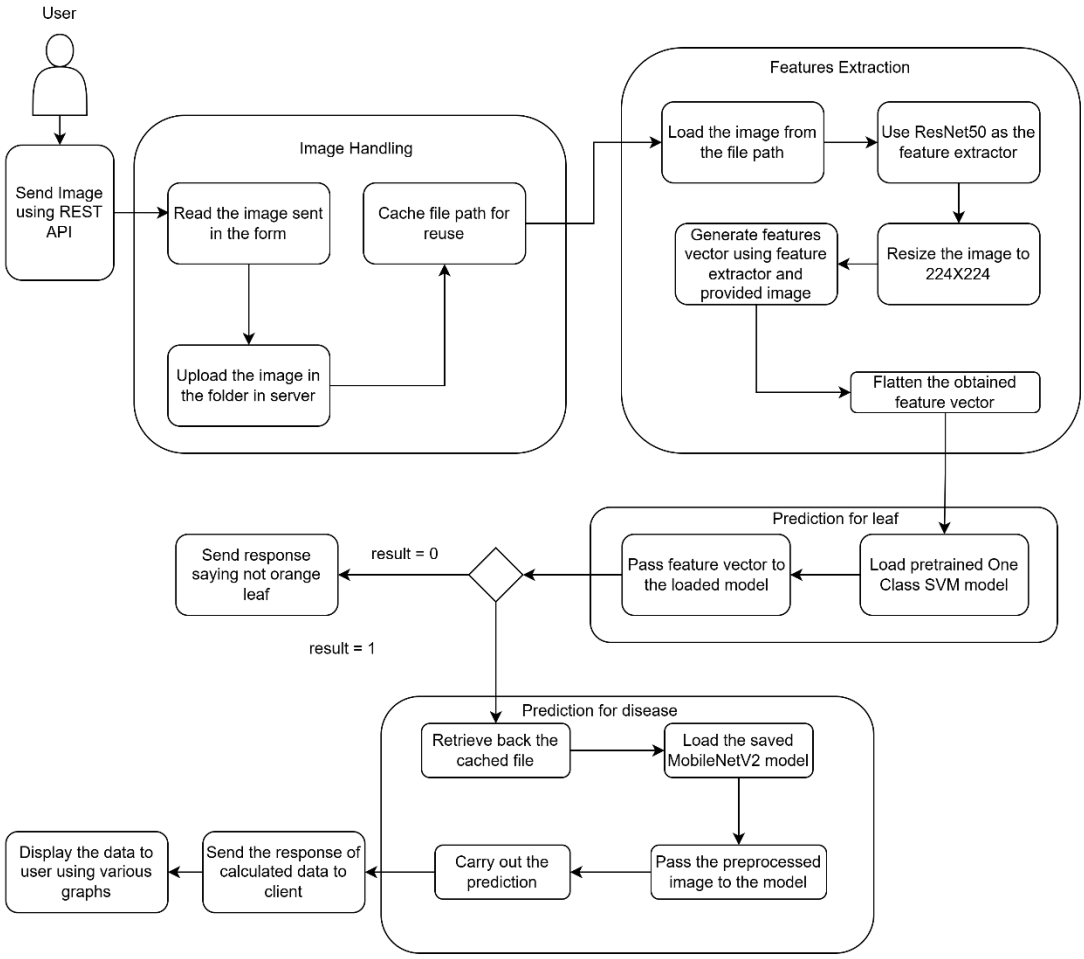


Figure 4.10: Block Diagram of Server side

4.4.1 Image Handling

The image is read and saved to the folder in the server. The file path of the saved image is cached for reuse.

4.4.2 Feature Extraction

The image is loaded from the cached file path and resized to the size 224 X 224. Normalization of the pixel values is done and the image data is flattened. The image data is passed to ResNet50 model from which the feature vector is obtained.

4.4.3 Prediction for Orange Leaf

Pre-trained One-Class SVM model is loaded and the extracted feature is passed to model. If the output generated is 1, it indicates that the passed image is of orange leaf. Otherwise, it indicates the image is not of orange leaf. If the result is 0, a direct JSON response is sent back to the client indicating that the image passed is not of the orange leaf and if the result is 1, redirection is carried out to another function for disease identification.

4.4.4 Disease Identification

The image is again received from the cached file path. The model developed using MobileNetV2 as the base model is then loaded to analyze the image. The prediction is obtained in array format which is then converted to the JSON format with disease and percentage value related to that disease and then sent back to the client.

5. IMPLEMENTATION DETAILS

5.1 Dataset Availability

The dataset used in this project for training of various model were obtained from the internet in which similar projects were conducted, titled "A Citrus Fruits and Leaves Dataset for Detection and Classification of Citrus Diseases through Machine Learning," is publicly available on the Data Mendeley website. This dataset can be accessed and downloaded for research purposes. It provides a comprehensive collection of images of citrus fruits and leaves, which are labeled with various diseases, including black spot, canker, greening, and scab, as well as healthy samples. The dataset was originally published as part of the research work by the authors of the paper and is an essential resource for developing and evaluating machine learning models for citrus disease detection. Moreover, local dataset was collected from orange orchard in Dhulikhel.

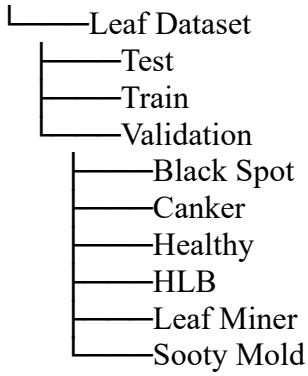
A sample of image from the dataset is shown in figure below:

Figure 5.1: Sample of online dataset (left) and local dataset (right)

5.2 Dataset Description

For the training of the model for citrus disease, total of 12000 images is used which contains 2000 images for each class. During the training process those images are divided

in the 0.7: 0.2: 0.1 ratio for training, validation and testing process. The folder structure of the dataset looks like:



The dataset used did not initially meet the MobileNetV2's specification for input dimension which requires the images to have the resolution of 224 X 224 pixels. In order to resolve this, generator function is used to preprocess the dataset by scaling every image to 224 X 224 pixels. As a result, the model is trained on a dataset that is complied with the input size specification, enabling precise and effective learning.

5.3 Dataset Augmentation:

Due to a smaller number of datasets, augmentation of the dataset is done with the help of the library named Augmentor. Images were rotated, zoomed, distorted, flipped (left-right and top-bottom) along with random distortion and increase of increase in brightness of the images were done to make the model learn images from different perspective. With the help of Augmentor library, the probability for different kinds of augmentation is set to get the images as needed. After the augmentation, file is stored in directory structure as shown above.

Table 5.1: Augmentation Details

Parameter	Values
Zoom	Probability = 0.3, min-factor = 0.8, max-factor = 1.5
Rotate	Probability=0.7, max-left-rotation=10, max-right-rotation=10
Flip to left and right	Probability = 0.5
Random Zoom	Probability = 0.5
Flip to top and bottom	Probability = 0.5
Random Brightness	Probability=0.4, min-factor=0.7, max-factor=1.2

5.4 Model Training

5.4.1 MobileNetV2

The dataset is first transformed to the required resolution of 224 X 224. The batch size was set to 32 with the shuffling of training dataset. The names of the directory were set to the name of the classes in the model. From the tensorflow, MobileNetV2 model was loaded as the base model. The layers of loaded model were frozen which prevents the value of the parameter to get updated during the training process. This is used to take advantage of the pre-trained reducing the computational cost and prevent the risk of overfitting. After that the model through which the training is done is made by adding MobileNetV2 as the base model followed by global average pooling layer, dropout layer and activation layer. The value of hyperparameters were set to be same as the base model. For the optimization, Adam optimizer was used. For the analysis binary cross entropy was used to calculate the loss and accuracy as a metric to analyze the model. The model was trained for 25 epochs which had the mean train time as 95s. The validation accuracy was calculated to be 97.44% and test accuracy was calculated to be 96.98%.

5.5 Mobile Application Development

The mobile application is designed and developed to help farmers in their citrus plantations. With the use of updated technology, the app facilitates users in predicting diseases found in citrus plants by checking the leaves. In addition, farmers can solve all related queries with the integrated chatbot, which comes with expert advice and more detailed information. This could be a complete tool for farming and disease management in order to improve productivity among citrus farmers.

5.5.1 Integration of Photo Capture & Server Communication

The developed application, built on Flutter, integrates a feature that enables users to capture photos and send them to a server for further processing. This functionality is designed to make data collection easier and ensure timely communication between the user's device and the backend server.

5.5.2 Photo Capture Feature

The application includes the camera module of the device, through which the user can take a photo within the application interface. This required the use of the image picker package, taking advantage of Flutter's well-maintained plugin ecosystem for seamless image capturing and storage. The captured images will be temporarily stored in the local directory of the app, then processed and prepared to be sent.

5.5.3 Data Transmission to Server

Once a photo is captured, the app formats the image data into a structured form suitable for server communication. The following steps outline the process:

Formatting of Data: The captured image is converted into a base64 string or uploaded as a file in a multipart/form-data format, depending on the server's requirements.

HTTP Request Preparation: The app creates an HTTP POST request, adding all necessary headers and body content. The request payload includes the image data and any additional metadata, such as timestamps, user identifiers, or geolocation tags.

Sending to Server: The application uses the http package to send the formatted request to the server endpoint. Error handling mechanisms is implemented to manage network issues or server-side errors gracefully.

5.5.4 Accessing Project Information

Moreover, the application offers its users a chance to view in great detail projects that are ongoing or completed. In this way, users get an opportunity to understand what the app is for and how it can relate to their needs. Users will be able to view descriptions, objectives, and outcomes of the projects, adding to transparency and engagement with what the app has on offer.

5.5.5 Cross-Platform Availability

Accordingly, the application targets a large section of the audience, as both Android and iOS versions are provided. In this case, cross-platform compatibility enables different device users to effectively make use of its benefits. Because the Flutter framework relies on a single codebase, the development and maintenance work are much easier and more efficient, ensuring complete consistency in user experience across multiple platforms.

5.5.6 Integration of Chatbot

The app includes more than just the photo-capture feature, but also provides an integrated chatbot to enhance the user's experience with intelligent and relevant responses. This chatbot is embedded right inside this application and has the following features:

User Guidance: It helps guide the end user through the app in using it to capture and then upload photos step by step.

Real-time Feedback: This chatbot sends real-time feedback to users upon successful photo uploads or to retry uploading in case something went wrong.

Knowledge Sharing: Leveraging its generative AI capabilities, chatbot offers insights and information related to citrus and its associated diseases. Users can query the chatbot about various citrus-related topics, including diseases like Huanglongbing (HLB), Blackspot, Leaf Miner, Sooty Mold and Canker. The chatbot provides accurate and relevant responses, equipping users with valuable knowledge to address specific concerns.

5.6 Base Model Deployment

The mobile application is designed with the server-based architecture, where the virtual machine (VM) in Azure is used for hosting the prediction model and the chatbot model. The server is responsible for processing the incoming request from the mobile application, executing the machine learning models and returning the results. Both the prediction model for the leaf prediction and disease detection along with the chatbot model are deployed within the VM, allowing the efficient handling of complex computation without burdening client devices.

When the HTTP request is received, the server extracts and decodes the request and route it to the web server which acts as an entry point for the requests. For the disease detection task, the request pass through the middleware inside the server which checks for the API key sent by the application inside the headers. After the matching of the API key, it starts to preprocess the image carrying out operations like resizing and normalization, to ensure it meets the required input format for the pre-trained prediction models. Then it carries out the prediction process. Once the prediction is made, the result is formatted into a structured response and sent back to the client as the HTTP response.

For the case of chatbot, the server processes the prompt sent from by the user where it checks if the user is asking about the same context related to previous prompts. On that basis it generates the appropriate response, embed it in the response and send it back to the client.

The architecture of the server is designed keeping scalability and efficiency in mind so that it can handle the request from many users from different places. Future enhancements can be improvement in model for the faster inference. This architecture also enables seamless updates to the prediction and chatbot response with the integration of advanced models without affecting the user experience.

5.7 Chatbot model

The two prominent approaches to enhance the performance of LLMs on low-frequent topics are: Retrieval Augmented Generation (RAG) and fine-tuning (FT) over synthetic data [26]. The findings indicate that while FT boosts the performance across entities of varying popularity, RAG surpasses FT by a large margin particularly for least popular factual knowledge. Additionally, the success of both RAG and FT approaches is amplified by improving retrieval and data augmentation techniques.

The chatbot developed for the project is a RAG based question-answer bot. For consistency and constancy of the context providing method, knowledge base is prepared as a PDF file which is read-only file. To answer the user query with the help of the knowledge base the generator (LLM) is adjusted in RAG system in following stages:

i. **Load PDF:** At first the prepared knowledge base is loaded as PDF with the help of a PDF loader native to langchain. The loader loads the pdf as a list of pages. Each page is stored as a document such that a document has a length equal to its number of pages. Each page document contains a metadata on which the source of the text along with its page number and the page label is stored.

ii. **Splitting Character from Text as Chunk:** LLMs cannot take into account a huge load of texts at once. So, a more efficient approach would be to create chunks of texts. A commonly used chunking strategy is Character Text Splitter from Langchain. The chunks are segregated based on a specific character. It can be a full stop; it can be a new line or any characters. Here which character is to be used as separator of chunks should be specified. This is aided by the number of characters to be allowed to chunked

at once. In order to not lose the context due to chunking, overlap is introduced. Once the chunks are created it is split into documents. Documents are a piece of text that contains metadata of the text piece.

iii. **Create Embeddings:** The chunks of text are converted into numerical representations called vectors or embeddings. The model used for creating embedding is sentence-transformers/all-MiniLM-L6-v2. "All-MiniLM-L6-v2" is a light weight model in sentence-transformers library with 30522 vocabularies. This model optimizes the memory as it maintains high performance even with its light weight. Each word, sub word and special words are mapped to their corresponding tokens from the vocabulary and hence the chunk of text is changed into vector representation which makes the human language understandable for the machine.

iv. **Store Embeddings as vector store:** The generated embedding is stored in a vector database for efficient retrieval. The FAISS vector database is used. FAISS is a library designed for efficient similarity search and clustering of dense vectors. Embeddings are stored in 2D array where row represents the embedding and number of columns represents dimension. The model "All-MiniLM-L6-v2" uses mean pooling to change token embeddings into a single chunk embedding. So, no matter the size of the chunk, the embedding is always of same size (384 dimensions in the case of All-MiniLM-L6-v2)

v. **Initialize the LLM model:** The chatbot needs a LLM model for proper and attractive text generation of text that contains the users answers as response. OpenAI, HuggingFaceHub, Gemini, Groq or locally loaded open source LLMs were the possible providers for the LLM. The project employs Groq's LLM which is right now the fastest API provider with a very high rate of output tokens. The LLM is initialized by importing the library. The API key is used to connect the bot's system to the LLM provider and from a list of provided models a model is selected. The project uses "llama-3.2-11b-vision", which is a collection of pretrained and instruction-tuned image reasoning generative models in 11B (text + images in / text out) with 10.6 billion trainable parameters. The Llama 3.2-Vision instruction-tuned models are

optimized for visual recognition, image reasoning, captioning, and answering general questions about an image. Llama 3.2-Vision is built on top of Llama 3.1 text-only model, which is an auto-regressive language model that uses an optimized transformer architecture. The tuned versions use supervised fine-tuning (SFT) and reinforcement learning with human feedback (RLHF) to align with human preferences for helpfulness and safety. It has 128k context length and its data volume is 6B (image, text) pairs.

vi. **Create retrieval for QA:** As retrieval is the brain of RAG system, the most processes and tasks happen in this stage. Accepting user query, passing it to embedding model, receiving the prompt embedding, passing the prompt to vector database, receiving the top ranked context results, passing the ranked result to a LLM (generator), receiving it and responding to user all happens due to retriever and thus it is the most important part of the system. Among the different types of retrievers, the project employs RetrievalQA which is a question-answering retrieval chain. A chain consists of sequential steps, conditional logic, or combinations of different operations to achieve a specific goal, such as answering a question or retrieving information. It accepts vector store as a retrieval, a LLM as generator and chain *keyword arguments* (kwargs) which may consist of prompt and memory methods as required. For remembering the context and past inputs the project employs a memory method for user experience and facilitating users for returning to past contexts.

vii. **Taking User Question:** The Question from the user is at first changed into embeddings or vector by sentence-transformers/all-MiniLM-L6-v2 model. The embedding captures semantic meaning of the question. The obtained embedding of the user question is then passed to the FAISS vector database where similarity search is conducted using Cosine similarity search. The stored embedding that has value near to 1 is then returned to the retriever. Generally, 4-10 embeddings are returned along with their text chunks unless specified in the kwargs. The formula used for calculation of Cosine Similarity search is:

$$Cosine\ Similarity = \frac{A.B}{||A||.||B||}$$ Equation: 5.1

where:

A.B = Dot product of vector

||A|| = Length of A

||B|| = Length of B

viii. **Generating and Providing Response:** The text chunks received after the similarity search along with the current context and user question is then fed into "llama-3.2-11b-vision" LLM model which processes the provided input and with the help of context, generates the necessary response by passing into the retriever which employs the model for text-generation. The generated response is provided to the retriever which contains metadata, query as well as response. From the generated response only, the answer is extracted and provided back to user.

ix. **Chatbot with Memory:** The five citrus diseases associated with the project are treated as individual topics. The topic detection method helps to identify the topic of user's current query by checking with predefined topics. The current topic determines the context of the chat. If current topic changes i.e. upon detection of new topic, context is reset. The model resets the context if current topic and previous topic are different. The context is created by summarizing user input and model output. It is stored until new topic is detected. This helps for continuity of the conversation with the chatbot. The main motive behind using this method is to create conversational chatbot rather than plain question-answer based chatbot. Moreover, resetting the context for each new topic helps to context relevant to current conversation. This method was employed as the provided memory method of LangChain could maintain the history of conversation but started to reply solely based on past context even when the context was shifted from a disease's topic to another. This resulted in hallucinations that was hard to treat. So, in order to resolve such issue, the concept of associated disease to be treated as separate topics were introduced.

5.8 Use Case Diagram

Use case Scenario name: Citrus Disease Detection and Prevention System **Participating actors:** User, Server **Preconditions:** Network connection is active User has installed mobile application in his device
Flow of events: User uploads Orange Leaf image to the server. Server processes the image and detects the disease Server analyzes the results through chatbot User sends query to the server. Server analyzes query through chatbot. Server sends results to user.

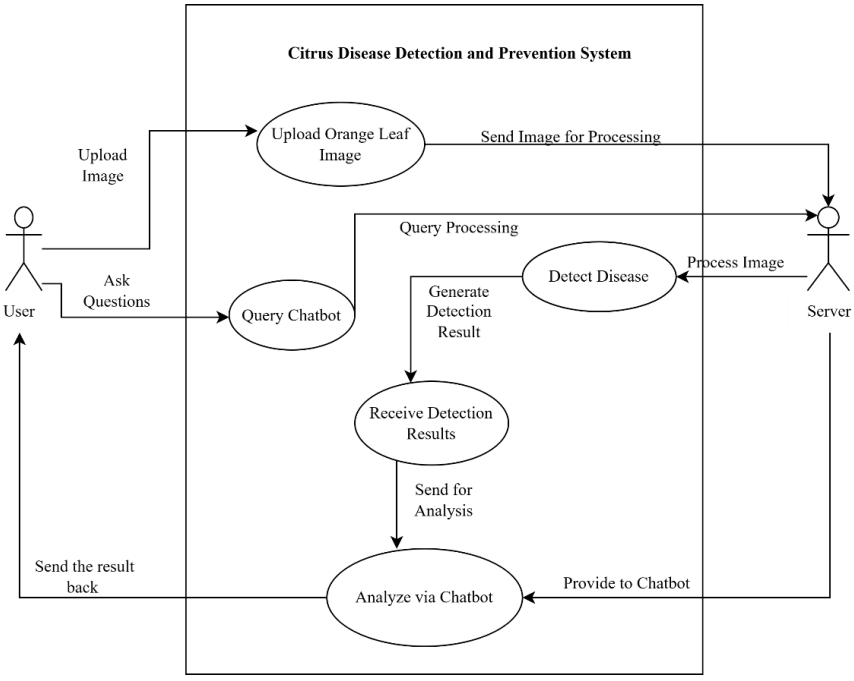


Figure 5.2: Use Case Diagram

6. RESULT AND ANALYSIS

6.1 Distinguishing between Leaves

The SVM model first distinguishes between the desired orange leaf vs non orange leaves and when the orange leaf is verified, the model passes it for further disease classification. The analysis of SVM model is given as follows:

6.1.1 Decision boundary on the projection

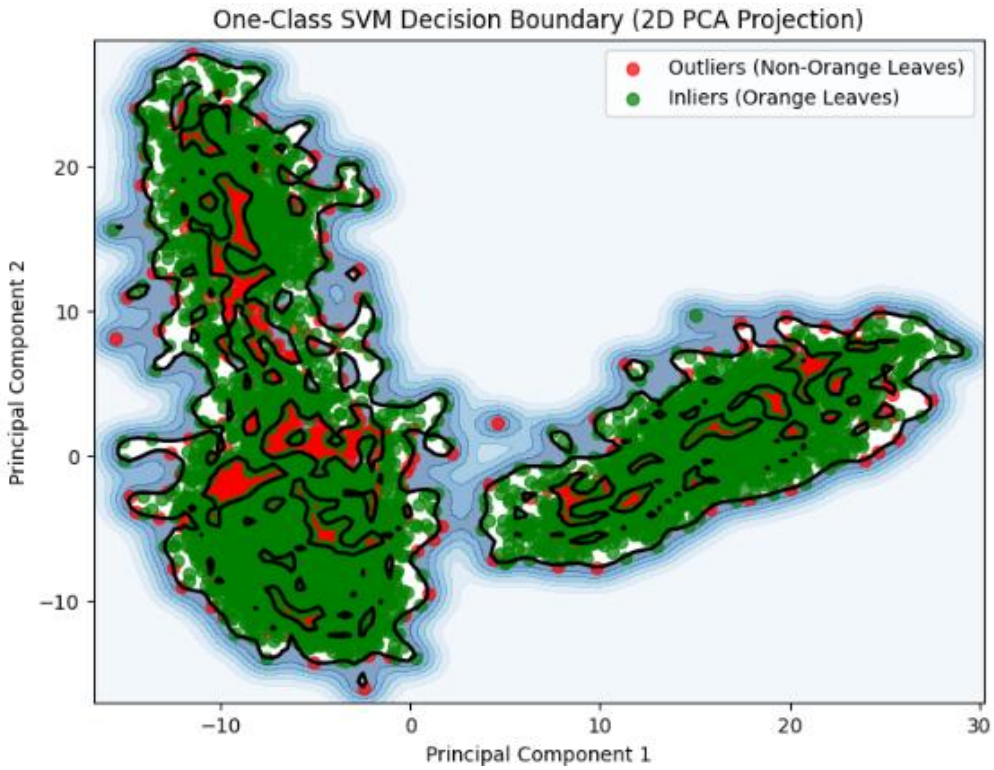


Figure 6.1: Decision Boundary of One-Class SVM

The decision boundary of a one-class SVM model trained on orange leaf image feature space is displayed in the image. The "inliers" the orange leaf photos that sit inside the learnt decision boundary are represented by the green points. Similarly, the "outliers" non-orange leaf photos that are outside the boundary are shown by the red pointers.

The One-Class SVM model has learned a complex representation of the orange leaf feature space, as shown by the decision boundary's uneven, non-linear shape. Because of this, it

can successfully differentiate orange leaves from other kinds of leaves or things. The distinct distinction between inliers and outliers indicates that, using the acquired feature representations, the model is very capable of identifying and differentiating between orange and non-orange leaves.

6.1.2 Confusion Matrix

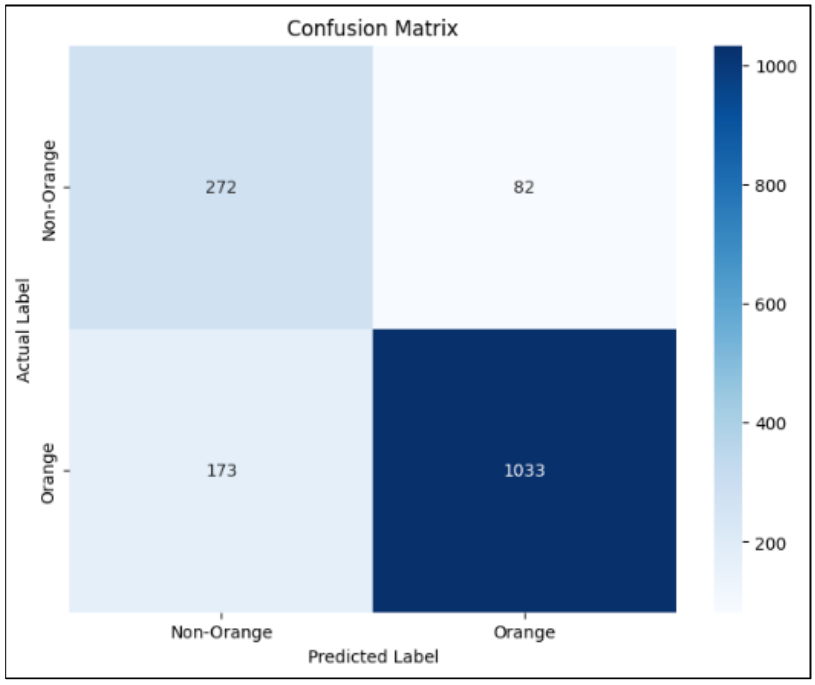


Figure 6.2: Confusion Matrix of One-Class SVM

The One Class SVM model effectively recognizes orange leaves, as shown by the confusion matrix, with 1,033 true positives and an accuracy of roughly 83.65%. However, it also displays 173 false negatives, suggesting that some orange leaves were mistakenly classified as non-orange. With only 82 false positives, the model's high precision of 92.55% indicates that it is probably right when it predicts an image to be an orange leaf.

Images of papaya and guava leaves fall into the non-orange group and are appropriately recognized. This contrast makes it clearer to us that some orange leaves were mistakenly classified as non-orange, which is the cause of the misclassifications.

6.1.3 Feature Space with train, test and additional image

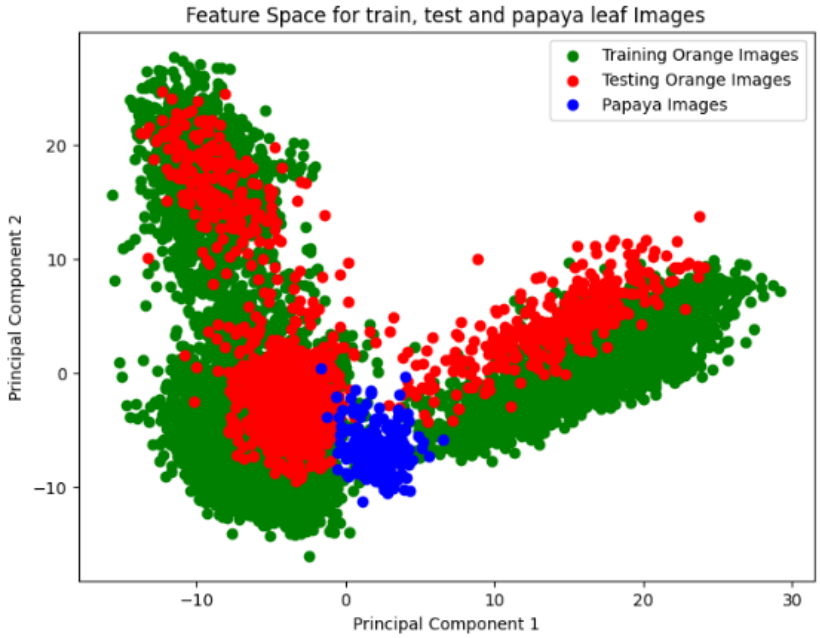


Figure 6.3: Feature Space with Train, Test and Additional Image

The distribution of training and test images for orange leaves and papaya leaves is displayed in the feature space plot. The papaya photos are represented by the blue dots, the testing orange images by the red dots, and the training orange images by the green ones.

The orange images used for training and testing are evenly distributed in the same area of the feature space, suggesting that the model has learned to recognize these similar characteristics. In order to categorize new, unseen orange leaf photos, the extracted features will be useful for generalizing the training data. The model appears to have learned to differentiate the visual characteristics of orange leaves from those of a different plant species, as evidenced by the papaya images clear separation from the orange leaf images. The model's capacity to distinguish orange leaves from other kinds of leaves is demonstrated by this separation.

The retrieved features have significant properties that can help with precise categorization of orange leaves versus other types of leaves, according to the feature space's general clustering and separation.

6.1.4 Feature Space of train and test dataset

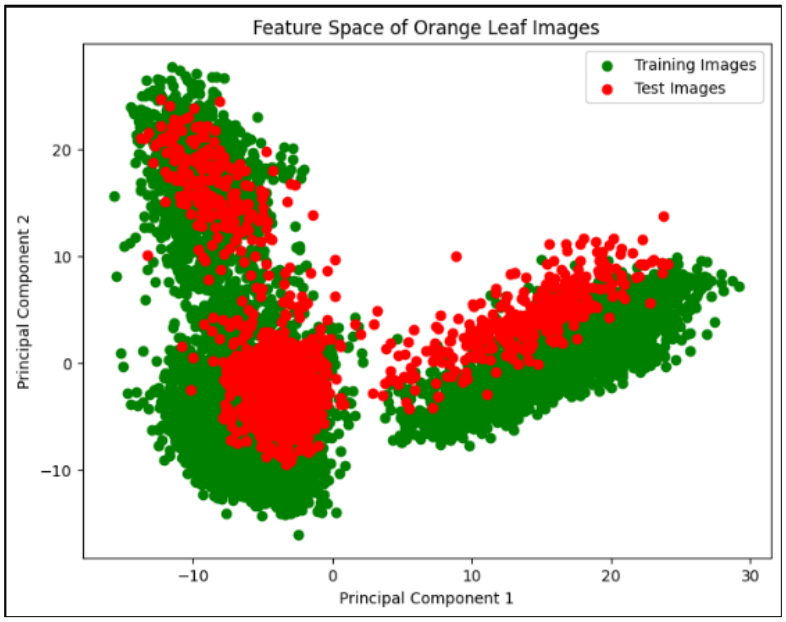


Figure 6.4: Feature Space of Train and Test Dataset

The orange leaf dataset's training and test picture distribution in principal component space is displayed in the feature space plot. The test images are represented by red dots, and the training images are represented by green dots. The features of the image are extracted using ResNet50 model which provides 2048 features at the end which is passed to OneClassSVM model later. The plot indicates that the model has successfully captured the distinctive qualities of the orange leaves by showing that the training and test data are well-separated in the feature space. This distinction in certain area in the plot implies that the model will be able to use the learned characteristics to correctly categorize new, unseen leaf images.

6.1.5 Histogram of Decision Scores

A bell-shaped distribution is revealed by the decision score histogram, indicating a common pattern in classification models. Positive and negative predictions are separated by the decision threshold, which is indicated by the vertical line. A higher frequency of lower decision scores is indicated by the distribution's slight rightward tilt. This skewness may indicate biases in the model's predictions or possible class imbalance. Additional information about the model's behavior and potential areas for development might be

obtained by examining the factors impacting the decision scores and analyzing the model's performance metrics, such as the confusion matrix and ROC curve.

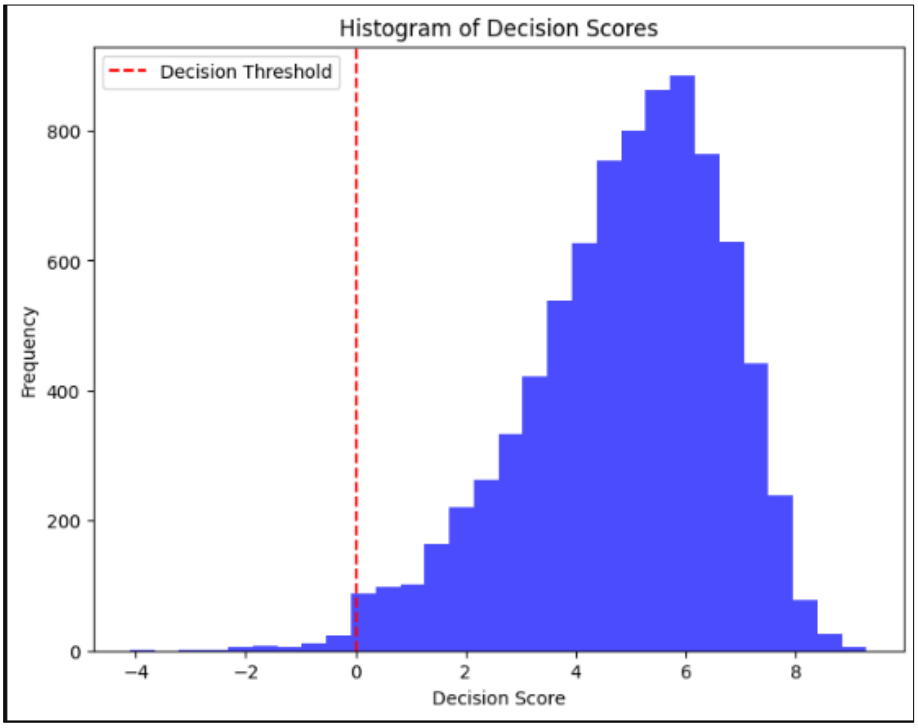


Figure 6.5: Histogram of Decision Scores of One-Class SVM

6.2 Comparisons of models

From InceptionNet, Inception-v3 was trained, from MobileNet, MobileNetV2 was trained and from EfficientNet, EfficientNet-B0 was trained. The models were trained to analyze their outputs separately. The results from each model are explained below:

6.2.1 Analysis of Inception-v3

The model was trained for 30 epochs, with a mean training time of 154 seconds per epoch. The mean training accuracy of the model was 0.96, and the validation accuracy was 0.96. The figure: 6.1 shows a detailed plot of training accuracy and validation accuracy at each epoch. The loss in training and validation is explained by the plot in figure:6.1.

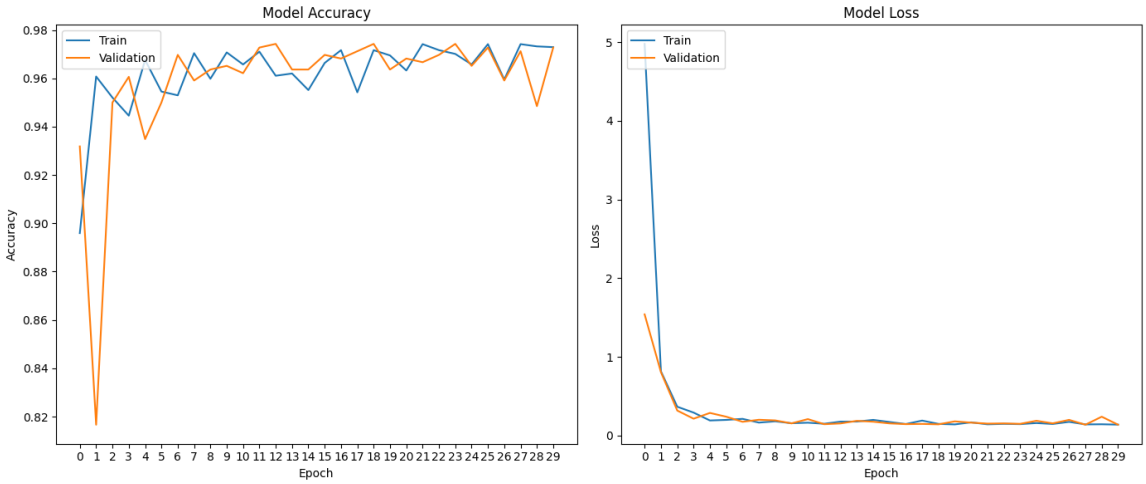


Figure 6.6: Accuracy and Loss Graph of Inception-v3

Upon evaluating the model, the test loss obtained was 0.1229, while the test accuracy was 0.9707. The model was then used to predict images from the test dataset and achieved an impressive accuracy of 0.9605. The table:6.1 shows the classification report of the model. Finally, for prediction visualization, a confusion matrix was plotted, as shown in figure:6.2. The model correctly identified 224 HLB leaves and 189 healthy leaves. However, the model incorrectly predicted 6 HLB-infected leaves as healthy and 11 healthy leaves as HLB-infected.

Table 6.1: Classification Report of Inception-v3

Classification Report

	precision	recall	f1-score	support
green	1.0	0.97	0.99	230
healthy	0.97	1.0	0.99	200
accuracy			0.99	430
macro avg	0.99	0.99	0.99	430
weighted avg	0.99	0.99	0.99	430

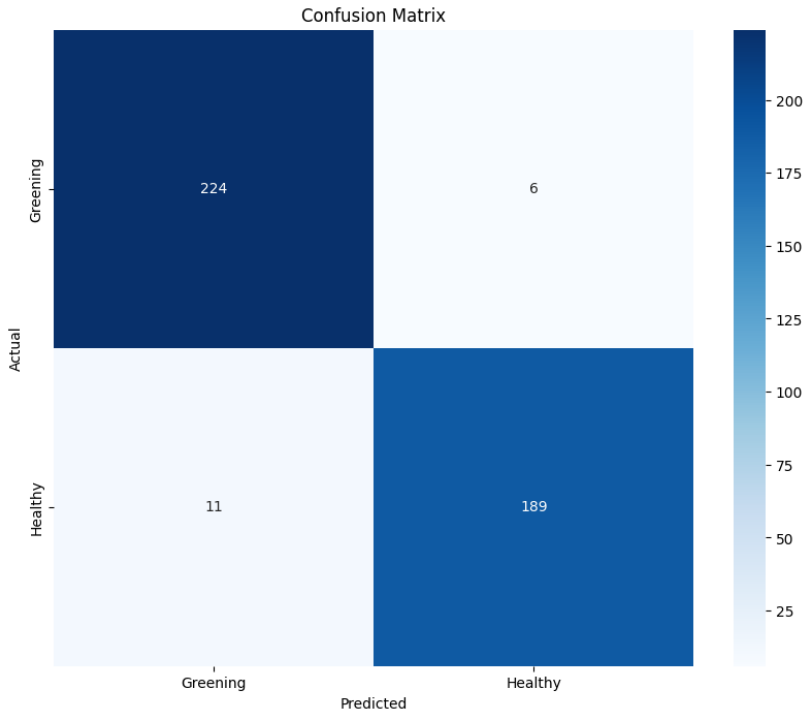


Figure 6.7: Confusion Matrix of Inception-v3

6.2.2 Analysis of EfficientNet-B0

The model was trained for 20 epochs, with a mean train time of 270.85 seconds. The mean training accuracy of the model was 0.961577 and the validation accuracy was 0.967515. The figure:6.3 shows the detailed plot of training accuracy and validation accuracy at each epoch. The loss in the training and validation are explained by the plot as in figure: 6.4. On the evaluation of the model, the test loss obtained was 0.0050 while the test accuracy was 0.9707. The model was then used to predict the given images form test dataset and provided an astonishing accuracy of 0.97. The table: 6.2 shows the classification report of the model. Finally for predictions visualizations, confusion matrix was plotted which is shown in figure:6.5. The model was able to correctly identify 413 HLB leaves and 382 healthy leaves. However, the model predicted 24 leaves to be healthy while they were HLB infected.

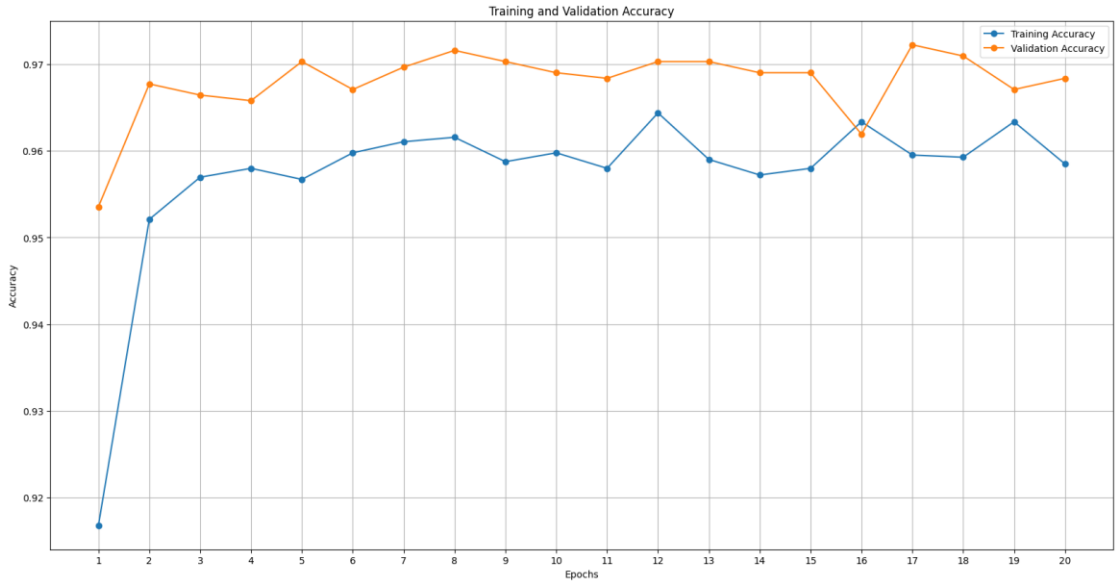


Figure 6.8: Accuracy vs Epoch Graph of EfficientNet-B0

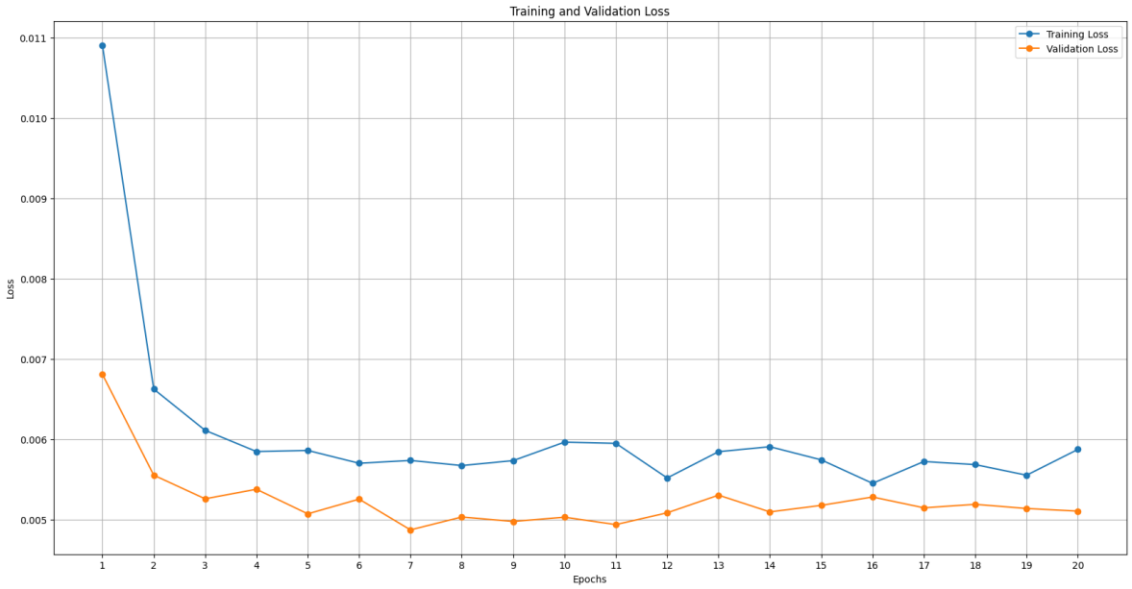


Figure 6.9: Loss vs Epoch Graph of EfficientNet-B0

Table 6.2: Classification Report of EfficientNet-B0

Classification Report

	precision	recall	f1-score	support
green	1.0	0.95	0.97	437
healthy	0.94	1.0	0.97	382
accuracy			0.97	819
macro avg	0.97	0.97	0.97	819
weighted avg	0.97	0.97	0.97	819

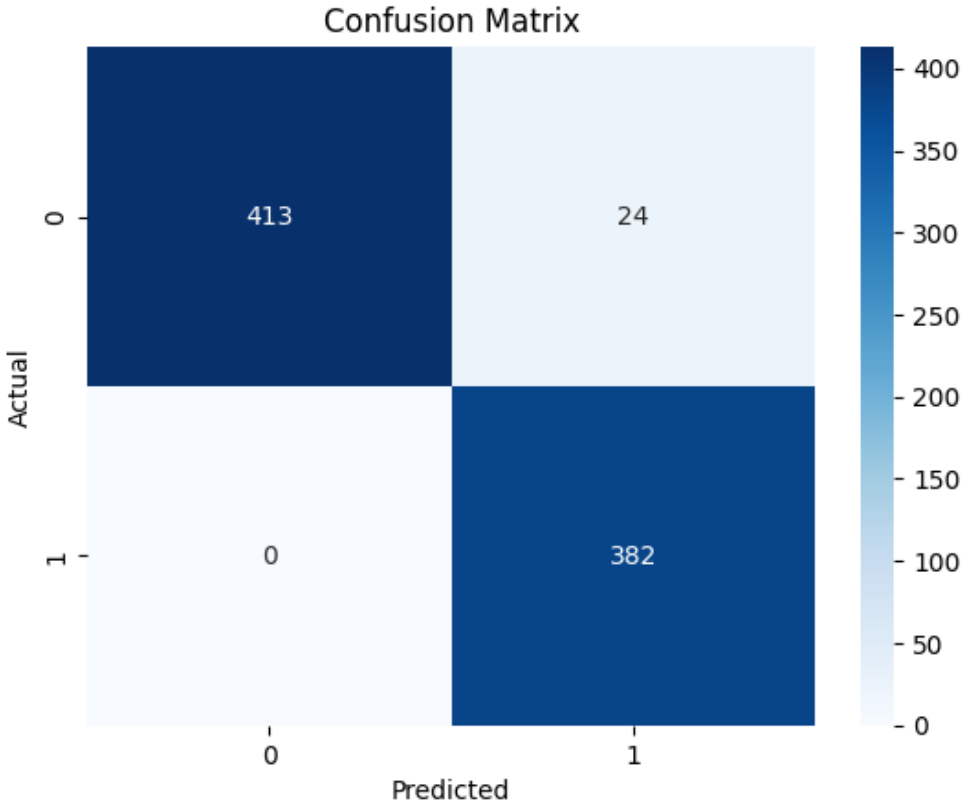


Figure 6.10: Confusion Matrix of EfficientNet-B0

6.2.3 MobileNetV2

The model was trained for 25 epochs which had the mean train time as 95s. The validation accuracy was calculated to be 91.39% and test accuracy was calculated to be 92.22%. Due to low weight of the model, it was deployed as the base model for the mobile application.

MobileNetV2 model is deployed in the mobile application after running for 25 epochs. After evaluation and testing, following observations and analysis are made:

6.3 ROC Curve of MobileNetV2

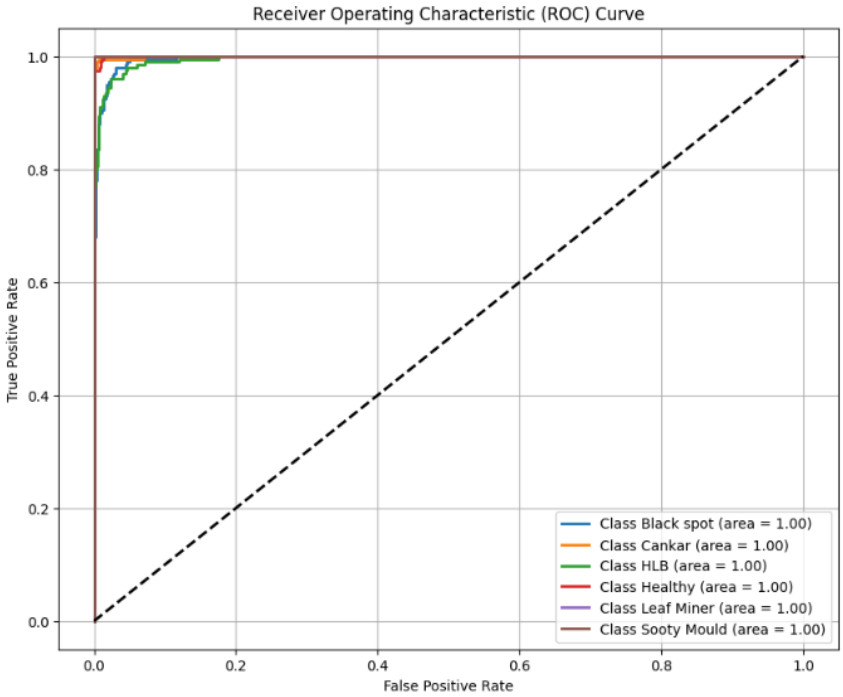


Figure 6.11: ROC Curve of MobileNetV2

The performance of the classification model is shown by the Receiver Operating Characteristic (ROC) curve for each of the six classes: Black Spot, Canker, HLB, Healthy, Leaf Miner, and Sooty Mold. Each curve has an area under the curve (AUC) value of 1.00 for every class, indicating nearly flawless classification performance. This suggests that for each category, the model maintains a very low false positive rate while achieving a high true positive rate. Excellent discrimination ability between positive and negative occurrences for each class is indicated by the high rise in the curves near the y-axis. The

model's performance, as demonstrated by the near-ideal ROC curves and AUC values, demonstrates its dependability and robustness in correctly classifying the provided classes.

6.4 Precision Recall Curve of MobileNetV2

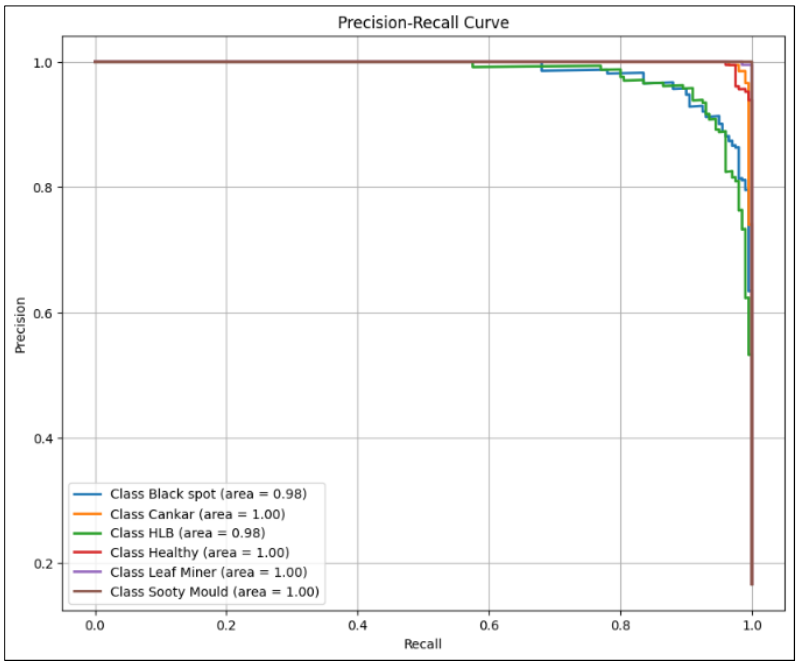


Figure 6.12: PR curve of MobileNetV2

The Precision-Recall (PR) curve examines the trade-off between precision and recall for each of the six classes in order to assess the model's classification performance. The model's remarkable ability to find a compromise between precision and recall is demonstrated by the high areas under the curve (AUC) for all classes: 0.98 for Black Spot and HLB, and 1.00 for Canker, Healthy, Leaf Miner, and Sooty Mold. The curves show that the model is dependable in retaining accurate predictions while capturing a significant percentage of actual positive cases, as evidenced by its excellent precision throughout a broad range of recall levels. The little variations in precision for HLB and Black Spot point to potential areas for additional model refinement. All things considered, the PR curve shows how well the model handles class imbalances and produces accurate predictions.

6.5 Confusion Matrix of MobileNetV2

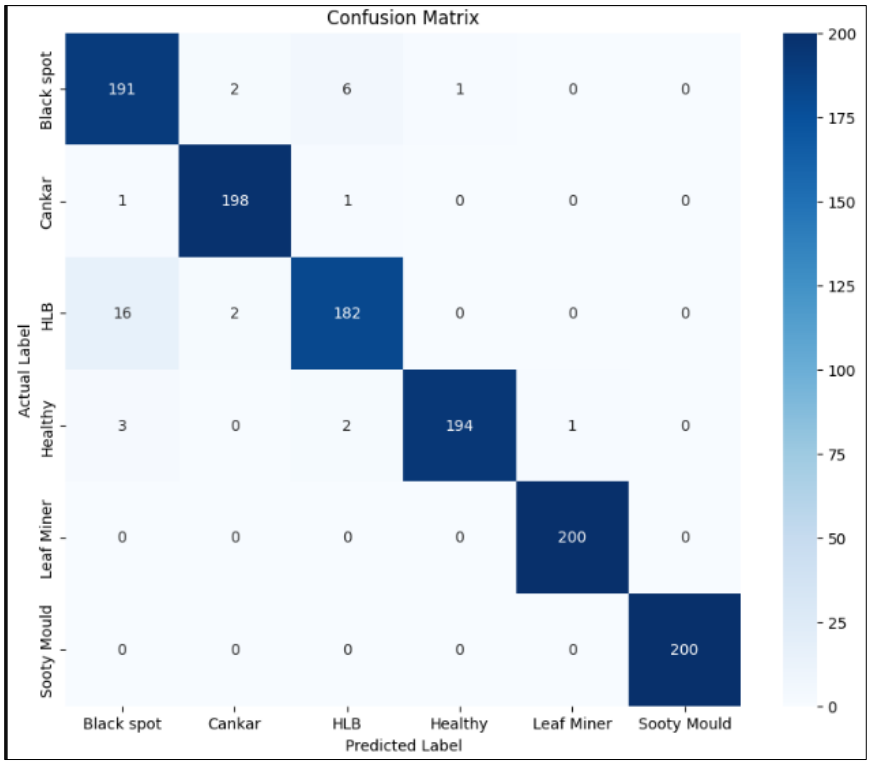


Figure 6.13: Confusion Matrix of MobileNetV2

With strong true positive rates along the diagonal, the confusion matrix demonstrates how well the model classifies the six classes: Black Spot, Canker, HLB, Healthy, Leaf Miner, and Sooty Mold. As an illustration of the model's reliability for these categories, it correctly recognized 191 Black Spot, 198 Canker, 182 HLB, 194 Healthy, 200 Leaf Miner, and 200 Sooty Mold occurrences. There are a few clear misclassifications, though, such some healthy samples being incorrectly labeled as HLB and Black Spot being mistaken for Canker or HLB. The model may find it difficult to differentiate between particular categories as a result of these inaccuracies, which could be caused by similarities in the dataset images.

6.6 Accuracy and Loss Plots of training and validation

During the training of a model, the plot shows the accuracy of training and validation as well as the loss across 20 epochs. While the validation accuracy shows a similar upward trend, convergent towards the training accuracy, the first graph displays the training

accuracy beginning at approximately 0.93 and progressively rising to 0.96. The training loss is shown in the second graph as starting high and rapidly declining in the first few epochs before reaching about 0.1 at the end of training. The validation loss follows a similar trend to the training loss, beginning higher and then decreasing over time.

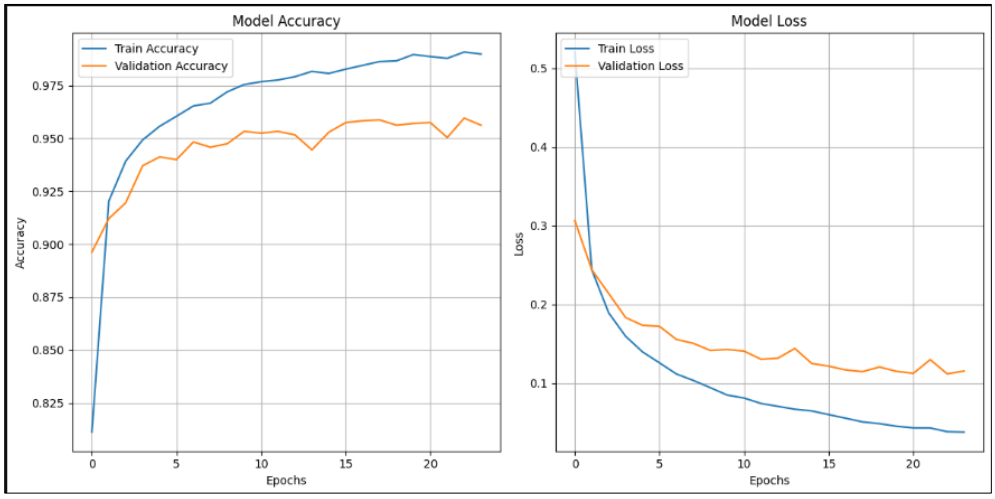


Figure 6.14: Accuracy and Loss plots of MobileNetV2

As seen by the increased accuracy and decreased loss on both the training and validation sets, these trends show that the model is successfully identifying the underlying patterns in the data. The model appears to be generalizing effectively and not overfitting to the training data, as indicated by the convergence of the training and validation metrics.

6.7 UMAP (Uniform Manifold Approximation and Projection) of MobileNetV2

Each data point represents a distinct leaf sample in the plot, which shows the dimensional reduction of the leaf data. The distinct class separation and grouping shows that the UMAP technique successfully preserved the underlying class structure while projecting the high-dimensional leaf attributes into a 2D space. This indicates that the algorithm has effectively identified the various forms of disease or leaf damage seen in the dataset. The model's capacity to identify the unique characteristics and patterns connected to the different leaf situations is demonstrated by the visual separation of the classes and their compactness.

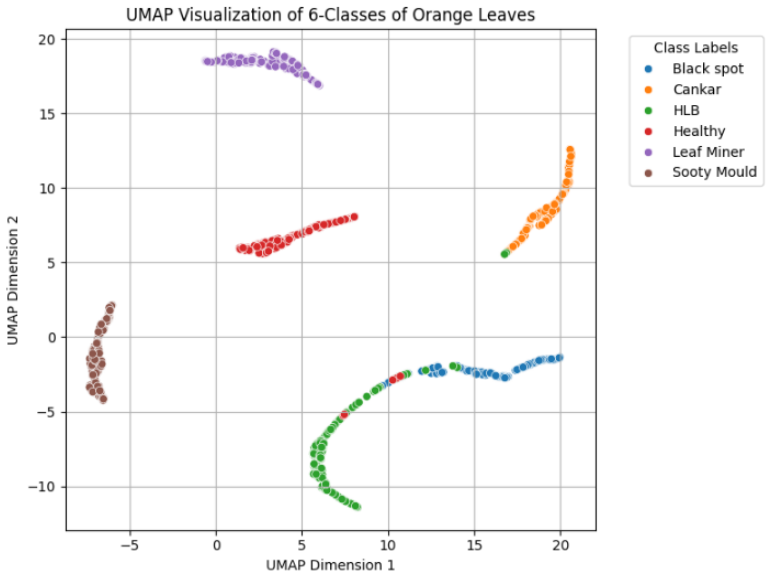


Figure 6.15: UMAP of MobileNetV2

6.8 Feature Map

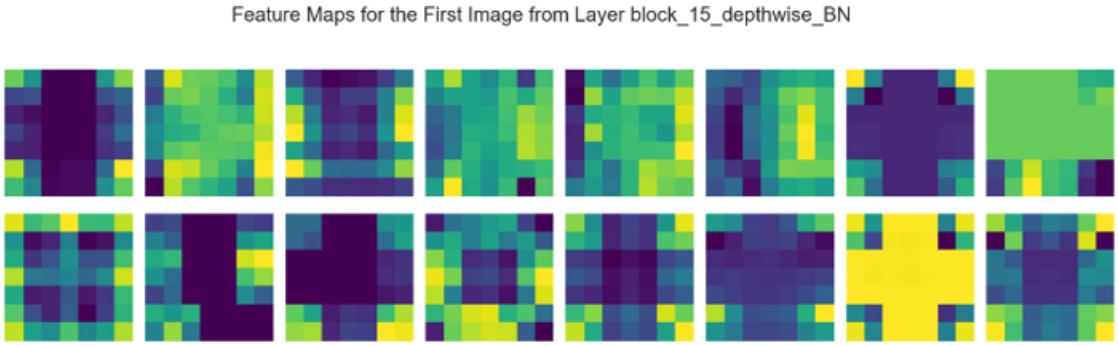


Figure 6.16: Feature Map of Layer block

It also shows the output of the first 10 filters among the 96 filters which the layer block_15_depthwise_BN contains. Each filter extracts different features from the image. One can observe several activation patterns in the feature maps, corresponding to different regions of importance detected by the network. The color scale changes from purple. Since the feature map is a direct result of training, it is clear that the network learned to put more emphasis on certain patterns due to the training data. The block_15_depthwise_BN layer captures computational patterns and textures in the input image, pointing out the model's ability to pay attention to relevant details. This will further enrich the model by diversifying training data for the network to learn generalized features. Moreover, dropout and other

techniques can be added to enhance generalization and avoid overfitting. All such improvements will further enhance the performance of the model to ensure a robust and accurate detection of citrus diseases in orange plants.

6.9 Classification Report of MobileNetV2

The classification report demonstrates how well the model detects a variety of plant illnesses. The overall accuracy percentage of 97% indicates that the disease classification procedure is very trustworthy. Precision, recall, and F1-score for Leaf Miner and Sooty Mould are all at 1.0, demonstrating the model's outstanding class-level performance. Canker's strong recall (0.97) and precision (1.0) allow it to perform exceptionally well as well, with an F1-score of 0.98. With a great recall (0.97) and precision (0.96), healthy samples rank second with an F1-score of 0.93. HLB has a little lower F1-score of 0.91 than Black Spot, which performs well with an F1-score of 0.93. Each class has an equal share of the dataset, which consists of 1200 testing samples. The model's balanced performance is confirmed by the precision, recall, and F1-score macro and weighted averages, all of which are 0.97.

Table 6.3: Classification Report of MobileNetV2

	Precision	Recall	F1-Score	Support
Black Spot	0.91	0.95	0.93	200
Canker	0.98	0.99	0.99	200
HLB	0.95	0.91	0.93	200
Healthy	0.99	0.97	0.98	200
Leaf Miner	1.0	1.0	1.0	200
Sooty Mould	1.0	1.0	1.0	200
Accuracy			0.97	1200
Macro Avg	0.97	0.97	0.97	1200
Weighted Avg	0.97	0.97	0.97	1200

6.10 t-SNE graph

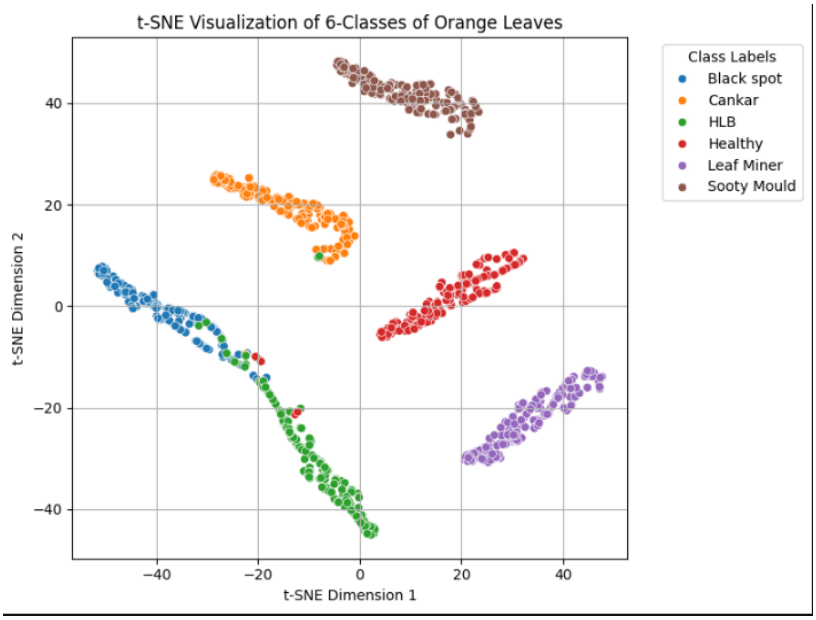


Figure 6.17: t-SNE graph of MobileNetV2

The six classes of orange leaves are clearly distinguished from one another as seen in the visualization. The model has successfully distinguished between the different forms of leaf damage or disease, as shown by the clear clustering of the data points for each class. The model appears to have captured the unique visual features of each condition, as evidenced by the close clustering within classes and the good separation between them. The model's excellent ability to distinguish and identify the many forms of orange leaf damage or illness found in the dataset is demonstrated by this visualization.

6.11 Chatbot Model

6.11.1 Comparison of Output of Chatbot Models

Multiple chatbot models were tested to be integrated into the mobile application. Initially, a chatbot tool named botpress was tried. The output from botpress was decent with average semantic accuracy of 0.7. However, the botpress model had a slow response and was inaccurate, so it was not considered for the project.

The fine-tuned Model of Alpaca and Gemma2 and RAG model using llama-3.2 were tested. RAG augments the prompt with the external data, while fine-Tuning incorporates the additional knowledge into the model itself [27]. These models were trained on the same dataset. They were then subjected to a semantic similarity test. Semantic similarity means how similar the semantic meaning of the obtained output is to the expected output.

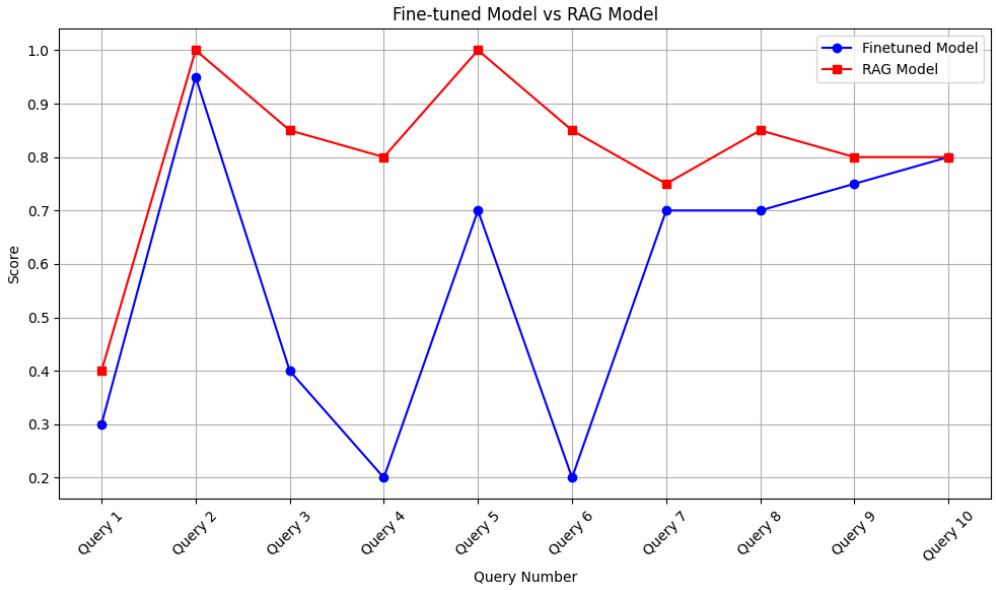


Figure 6.18: Fine-tuned model vs RAG model

The graph suggests that the RAG model has outperformed the fine-tuned model when a semantic similarity of these models is compared to each other. Moreover, the size of the fine-tuned model was very large and the response speed was slow compared to the RAG model. So, RAG model using llama-3.2-11b-vision is used for the project.

6.11.2 Output of RAG model

The RAG model was trained on dataset created for the project that contains the information related to citrus diseases in Oranges and their preventions in pdf format. The model was then tested with 10 random questions. Various metrics were calculated for the model.

The model has average precision and average recall of 0.95 which shows how efficiently the model is answering the queries. The average semantic similarity is 0.85 which suggests the output generated by the model is nearly similar to the expected output.

Table 6.4: Evaluation Metrics for Chatbot

Metrics	Value
Average Precision	0.95
Average Recall	0.95
Average semantic similarity	0.85

Moreover, the bar graph obtained from semantic similarity score is shown below. For the direct question which are related to citrus diseases, the results are very accurate. For indirect questions such as “What are its symptoms?”, the results are nearly accurate because the model starts to hibernate as the context starts to pile up. The query with lowest similarity score was totally unrelated to the diseases so score is quiet low. This shows how efficient the RAG chatbot is for queries related to the citrus diseases.

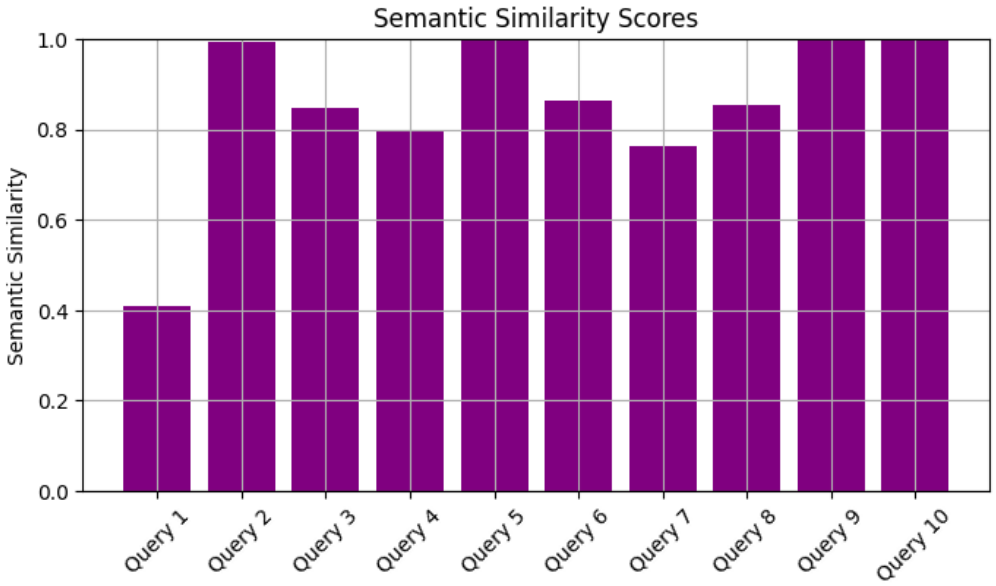


Figure 6.19:Barchart showing Semantic Similarity Score

6.12 Mobile Application

The mobile app developed using Flutter for orange disease detection demonstrated promising results during testing. The app was tested on various devices, ensuring compatibility and consistent performance across different platforms. The user interface, built with Flutter, provided an intuitive and seamless experience, making it easy for users to capture and submit images for analysis.

The disease detection model showcased high accuracy in identifying infected orange leaves. The server processing ensured rapid analysis, with the average image processing time being under five seconds. This real-time feedback is crucial for users in the agricultural sector who require quick and reliable results to take immediate action.

In addition to its accuracy and speed, the app's lightweight nature allowed it to run efficiently on devices with limited processing power and memory. Overall, the app's performance in real-world scenarios validated its effectiveness as a tool for detection and management of Huanglongbing in citrus crops.

Here are the snapshots of the apps:

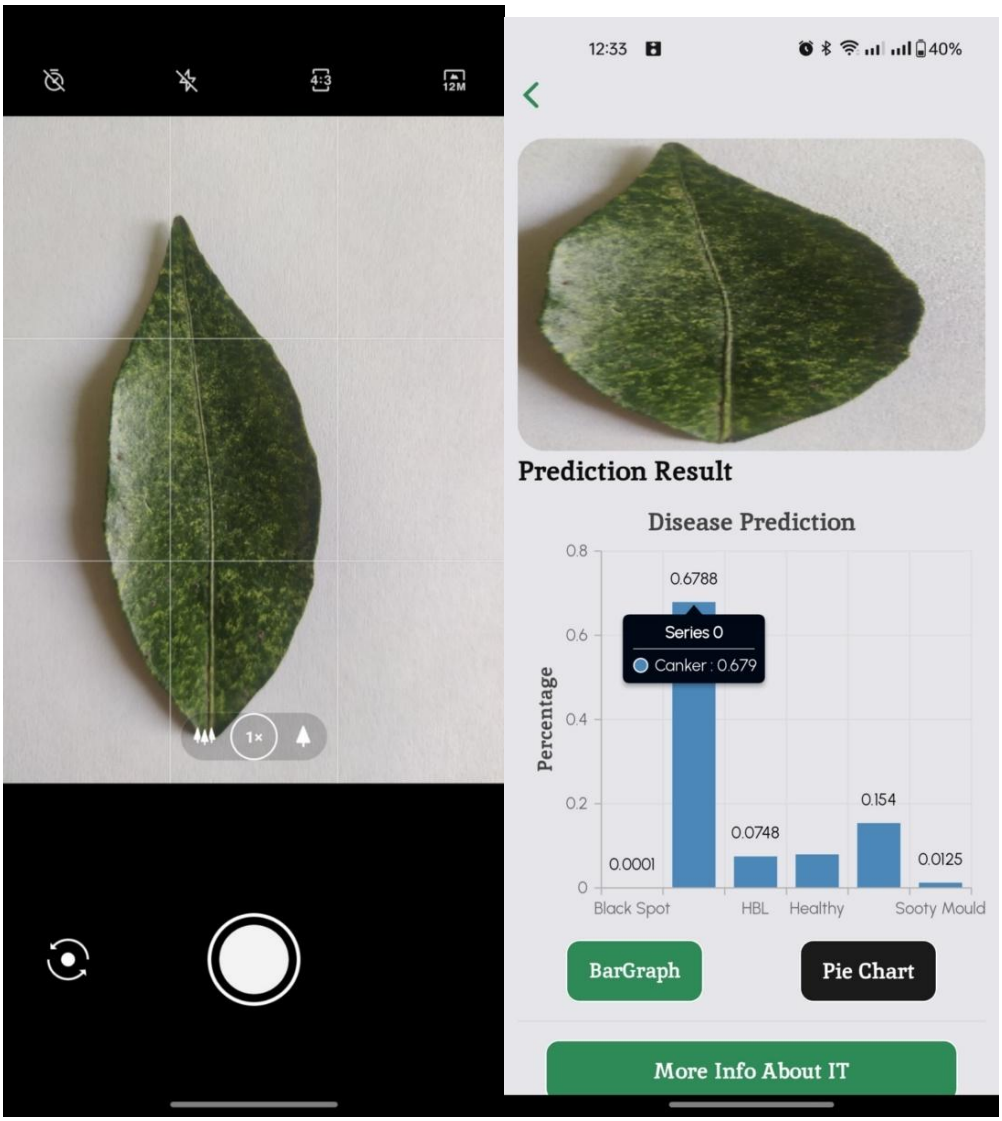


Figure 6.20: Mobile Application Displaying Output

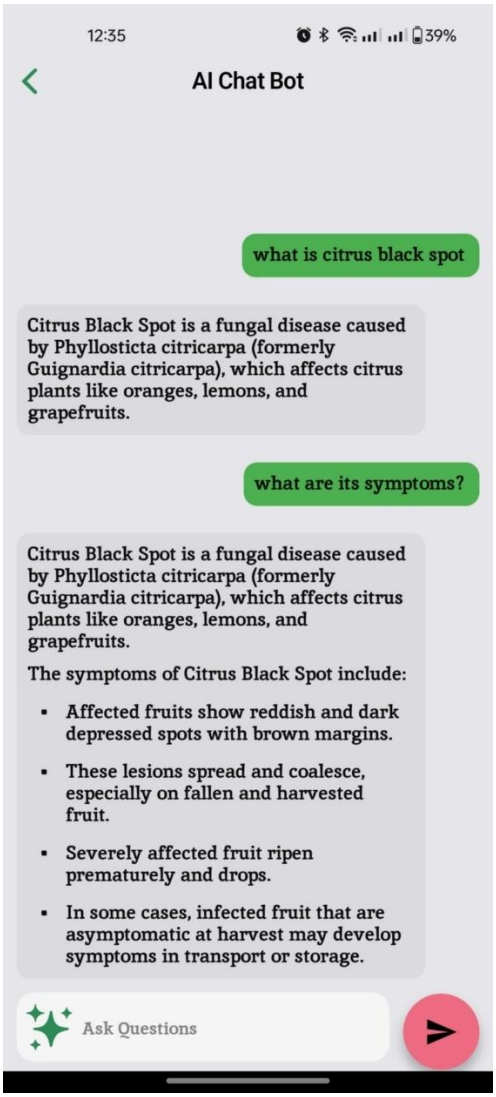


Figure 6.21: Chatbot

7. FUTURE ENHANCEMENTS

- Enhancing the User Experience by a subtle analysis of the provided leaf and providing seamless connection to chatbot feature from detection feature
- Real-time disease detection that will predict the disease with a confidence score as soon as a leaf is placed in-front of the device
- Addition of feature where diseases' severity can be assessed using more advanced methods.
- There can be integration with IoT devices aside from mobile app such that continuous monitor can be made possible [28].
- Improving the UI as well as how the data of user input and output is handled, tightening any loose ends
- Adding Nepali fonts to enhance User Experience
- Adding extra features like temperature, humidity, notices and articles to aid farmers about the governmental aids in agriculture.

8. CONCLUSION

With the combination of prediction model with deep learning and a chatbot, this project effectively provides a comprehensive system for the identification and prevention of citrus diseases in oranges. In order to guarantee that the disease detection model runs on relevant data, the system integrates an initial image classification model that reliably separates orange leaves from other plant leaves.

The core of system utilizes a CNN model with two convolutional layers after MobileNetV2 as the foundation model. It was able to classify six classes with five citrus disease including Black Spot, Canker, HLB, Healthy, Leaf Miner, and Sooty Mold with an accuracy of 95.6%. Additionally, the system uses a One Class SVM model to recognize orange leaves, with an accuracy of 85.6% in test orange leaves. This high accuracy of the model empowers the farmers and producers to make informed decision about the condition of citrus fruits in their orchards.

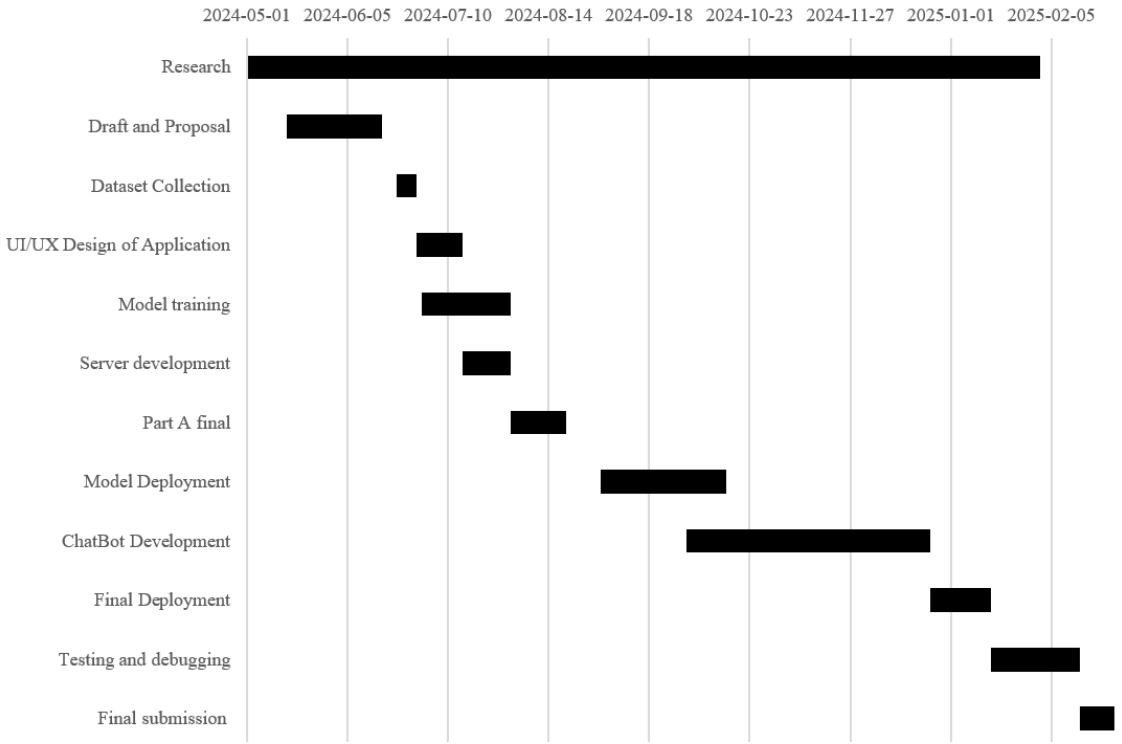
The user-friendly chatbot interface and excellent accuracy enable farmers and growers to make knowledgeable decisions about disease prevention and management. The chatbot works as the valuable knowledge resource addressing the general queries that farmers may have related to citrus and diseases that can affect it. The server-side architecture centralizes all the heavy processing task enhancing the reliability and efficiency.

While the current system demonstrates the promising result, future work could focus on real time detection for the disease and its severity along with the integration with IoT devices. By the refinement of the model and expansion of its capabilities, this system can play a significant role in mitigating the impact of citrus diseases and ensuring the enhancement in citrus production.

9. APPENDICES

Appendix A: Project Timeline

Table 9.1: Gantt Chart showing Project Schedule



Appendix B: Project Budget

The following table shows the total expenditure:

Table 9.2: Project Expenditure

Items	**Price (NRs.)**
Hosting Server	Rs. 4000
Data Collection	Rs. 4000
Printing	Rs. 4000
Field Visit	Rs. 4000
Total	Rs. 16000

Appendix C: Field Visit

Figure 9.1: Meeting with Dr. Umesh Acharya

Figure 9.2 : Collection of Local Dataset

Figure 9.3: Meeting at District Agriculture Development Office, Kavre.

Figure 9.4: Mobile app evaluation at Warm Temperate Horticulture Center, Kirtipur

References

[1] K. Timilsina, "Citrus Greening: Nepal's groves under threat," *ResearchGate,* 15 April 2019.

[2] N. Dhakal, D. Adhikari, K. Subedi, S. A., D. Tiwari, A. Bajracharya and S. Humagain, "ASIAN CITRUS PSYLLID Diaphorina citri (Kuwayama) (Hemiptera: Liviidae) AND ITS DETECTION SURVEY IN CITRUS ORCHARDS OF SINDHULI, NEPAL," *International Journal of Applied Sciences and Biotechnology,* vol. 7, 2022.

[3] E. Shahbaz, M. Ali, M. Shafiq, M. Atiq, M. Hussain, R. M. Balal, A. Sarkhosh, F. Alferez, S. Sadiq and M. A. Shahid, "Citrus Canker PAthogen, Its Mechanism of Infection, Eradication and Impacts," *Plants,* vol. 12, no. 1, p. 123, 2022.

[4] "Sooty Mold / Home and Landscape / UC Statewide IPM Program (UC IPM)," Ucanr.edu, 2015.

[5] E. Grafton-Cardwell, K. Godfrey, D. Headrick, P. Mauk and J. Pena, "Citrus Leafminer and Citrus Peelminer," April 2008.

[6] H. G. Toessi, D. Amari, R. Sikirou and D. Kone, "First report of citrus black spot disease caused by Phyllosticta citricarpa in Benin," *New Disease Reports,* vol. 47, no. 1, January 2023.

[7] R. Giri, Biotechnologist at Warm Temperate Horticulture Center, NCFD,Kirtipur, 2025.

[8] O. Almutiry, M. Ayaz, T. Sadad , I. U. Lali, A. Mahmood and N. U. Hassan, "A novel framework for multi-classification of guava disease," *Computers, Materials & Continua,* vol. 69, no. 2, pp. 1915-1926, 2021.

[9] A. Khattak, M. Asghar, U. Batool, M. Z. Asghar, H. Ullah and M. Al-Rakhami, "Automatic Detection of Citrus Fruit and Leaves Diseases Using Deep Neural Network Model," *IEEE Access,* vol. 9, pp. 112942-112954, 2021.

[10] L. Li, S. Shang and B. Wang, "Plant Disease Detection and Classification by Deep Learning—A Review," *IEEE Access,* vol. 9, pp. 56683-56698, 2021.

[11] R.-Z. Qiu, S.-P. Chen, T. Huang, G.-C. Fan, M.-X. Chi, R.-B. Wang, J. Zhao and Q.-Y. Weng, "An automatic identification system for citrus greening disease (Huanglongbing) using a YOLO convolutional neural network," *Front. Plant Sci,* vol. 13, 20 December 2022.

[12] B. Kaur, T. Sharma, B. Goyal and A. Dogra, "A Genetic Algorithm based Feature Optimization method for Citrus HLB Disease Detection using Machine Learning," *IEEE Xplore,* pp. 1052-1057, 2020.

[13] X. Deng, Y. Lan, T. Hong and J. Chen, "Citrus greening detection using visible spectrum imaging and C-SVC," *Computer and Electronics in Agriculture,* November 2016.

[14] D. Yang, F. Wang, Y. Hu, Y. Lan and X. Deng, "Citrus Huanglongbing Detection Based on Multi-Modal Feature Fusion Learning," *Front. Plant Sci.,* vol. 12, 23 December 2021.

[15] H. Al-Hiary, S. Bani-Ahmad, M. Ryalat, M. Braik and Z. Alrahamneh, "Fast and Accurate Detection and Classification of Plant Diseases," *International Journal of Computer Applications,* vol. 17, no. 1, 2011.

[16] S. R. Dubey and A. S. Jalal, "Adapted Approach for Fruit Disease Identification using Images," *International Journal of Computer Vision and Image Processing (IJCVIP),* vol. 2, no. 3, pp. 44-58, 2012.

[17] G. Geetharamani and J. A. Pandian, "Identification of plant leaf diseases using a nine-layer deep convolutional neural network," *Computers & Electrical Engineering,* vol. 76, pp. 323-338, June 2019.

[18] M. Chohan, A. Khan, R. Chohan, S. H. Katpar and M. S. Mahar, "Pant Disease Detection using Deep Learning," *International Journal of Recent Technology and Engineering,* vol. 9, no. 1, pp. 909-914, May 2020.

[19] A. A. Ahmed and G. P. Reddy, "A Mobile-Based System for Detecting Plant Leaf Diseases Using Deep Learning," *AgriEngineering,* pp. 478-493, 2021.

[20] Y. P. K. Q. J. K. M. P. Q. D. M. M. a. R. M. A. Burks T., "Automated Classification of Citrus Disease on Fruits and Leaves Using Convolutional Neural Network Generated Features from Hyperspectral Images and Machine Learning Classifiers," *Journal of Applied Remote Sensing,* vol. 18, no. 1.

[21] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L. C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR),* pp. 4510-4520, 2018.

[22] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *International Conference on Machine Learning,* 2019.

[23] C. Szegedy, V. Vanhouce, S. Loffe, J. Shlens and Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," *Computer Vision and Pattern Recognition,* 2015.

[24] R. e. a. Toari, "Alpaca: A Strong, Replicable Instruction-Following Model," 2023.

[25] G. T. e. al., "Gemma 2: Improving Open Language Models," 2024.

[26] H. Soudani, E. Kanoulas and F. Hasibi, "Fine Tuning vs. Retrieval Augmented Generation for Less Popular Knowledge," in *roceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*, ACM, 2024, pp. 12-22.

[27] A. Balaguer, V. Benara, R. L. Cunha and T. Henry, "RAG vs Fine-tuning: Pipelines, Tradeoffs, and a Case Study on Agriculture," arXiv, 2024.

[28] R. R. Lamsal, P. Karthikeyan, P. Otero and A. Quintana, "Design and Implementation of Internet of Things (IoT) Platform Targeted for Smallholder Farmers: from Nepal Perspective," *Agriculture,* vol. 13, no. 10, 2023.